D-HNSW: Deterministic Hierarchical Navigable Small World Graphs for Verifiable Nearest Neighbor Search

Hong-Son Nguyen University of Transport and Communications (UTC)
Hanoi, Vietnam
Email: sonlearn155@gmail.com

***Abstract*—Approximate nearest neighbor (ANN) search is central to modern data systems, yet the dominant HNSW algorithm relies on IEEE 754 floating-point arithmetic, introducing platform-dependent nondeterminism incompatible with blockchain consensus and verifiable AI. We present D-HNSW (Deterministic HNSW), which guarantees bit-exact reproducibility by replacing floating-point distance computations with Q32.32 fixed-point integer arithmetic and seeding graph construction with Keccak-256 PRNG. Our fixed-point representation achieves errors below 4×10^{-10} relative to `f64`. Measured scalar overhead ranges from $5.56\times$ (1536-d) to $9.62\times$ (96-d); with projected AVX2 SIMD acceleration, this reduces to $1.39\times$–$2.41\times$. Quantitative comparison shows D-HNSW cuts on-chain verification cost by 5–$10\times$ versus ZK-SNARKs while providing instant finality. Experiments on seven benchmarks (96–1536 dimensions, including OpenAI embeddings) confirm identical recall to standard HNSW, with determinism verified via SHA-256 hashing across x86-64 and ARM64 architectures and validated on SIFT-1M at scales up to 200K vectors achieving 97.81%–99.77% Recall@10.**

***Index Terms*—Approximate nearest neighbor search, deterministic computation, fixed-point arithmetic, HNSW, blockchain, verifiable AI, vector databases**

I. Introduction

APPROXIMATE nearest neighbor (ANN) search is central to modern data-intensive systems, from information retrieval [1] and recommendation engines to the retrieval-augmented generation (RAG) pipelines behind large language models. As vector databases scale to billions of entries [2], the pressure to balance recall with throughput has driven extensive algorithmic development. Among tree-based methods [3], locality-sensitive hashing [4], product quantization [1], and graph-based approaches [5], the Hierarchical Navigable Small World (HNSW) algorithm [6] has become the dominant choice, delivering strong recall-throughput tradeoffs across standard benchmarks [2] and powering production systems including Faiss [7], Qdrant, Weaviate, and Milvus.

Yet HNSW has a limitation that is often overlooked: *nondeterminism*. Standard implementations use IEEE 754 floating-point arithmetic for distance computation, which does not guarantee identical results across hardware platforms, compiler settings, or instruction set architectures [8]. Floating-point nondeterminism arises from multiple sources: (1) the availability and use of fused multiply-add (FMA) instructions, which compute $a \times b + c$ with a single rounding step rather than two; (2) compiler-driven instruction reordering that exploits algebraic properties not preserved under finite-precision arithmetic; (3) variations in SIMD lane widths (SSE, AVX2, AVX-512, NEON) that change the order of partial-sum accumulation; and (4) transcendental function implementations that differ across math libraries. These seemingly minor discrepancies can propagate through the greedy graph traversal of HNSW, where a single tie-breaking difference in distance comparison may redirect the search path, potentially yielding entirely different result sets.

For most applications this is harmless—a search engine returning slightly different results on different servers is acceptable. However, several emerging application domains *require* deterministic, verifiable computation:

- **Blockchain and smart contracts.** Decentralized systems require all validator nodes to reach identical state transitions from identical inputs [9], [10]. A vector similarity search embedded in a smart contract must produce bit-exact results regardless of the validator's hardware, or consensus will fail. Cardano's EUTXO model explicitly mandates deterministic transaction evaluation [11], and integer arithmetic is widely recognized as essential for blockchain consensus [12].
- **Verifiable and auditable AI.** As AI systems are deployed in regulated domains—healthcare diagnostics, financial credit scoring, autonomous vehicles—regulators increasingly demand that inference results be reproducible and auditable [13]. Floating-point nondeterminism undermines this requirement, as demonstrated by recent work on nondeterminism-aware verification [14].
- **Decentralized vector databases.** Emerging Web3 architectures for similarity search [15], [16] and AI-powered data trading [17] require that query results be independently verifiable. ANNProof [18] addresses verification at the proof level but does not eliminate the underlying nondeterminism in distance computation.
- **Deterministic AI infrastructure.** Projects such as Valori [19] and EigenAI [13] are building deterministic substrates for AI computation, recognizing that reproducibility is a prerequisite for trust in distributed AI systems.

In this work, we propose D-HNSW, a variant of the HNSW algorithm designed to ensure bit-exact result consistency across diverse hardware platforms. The core idea is to employ 64-bit fixed-point integer arithmetic in place of IEEE 754

Manuscript received April, 2026.

floating-point distance computations, thereby producing identical results on any processor that supports standard integer operations—which encompasses virtually all modern hardware. Our main contributions are as follows:

1) **Fixed-point distance computation framework.** We design a Q32.32 fixed-point arithmetic system with saturating operations, supporting squared Euclidean distance, Euclidean distance (via Newton–Raphson integer square root), and cosine similarity. We provide formal error bounds showing relative errors below 4×10^{-10} compared to IEEE 754 double precision (Section IV), grounded in the Determinism Trilemma framework (Section III-D).
2) **Performance optimization suite.** We introduce five complementary optimizations—SIMD-accelerated fixed-point operations, two-phase distance computation, neighbor reordering, early termination, and partial distance pruning—that collectively reduce the naïve scalar fixed-point overhead from 6.7×–9.4× to 1.52×–2.44× with AVX2 SIMD acceleration (Section V).
3) **Deterministic graph construction.** We replace all sources of nondeterminism in the HNSW index construction process, including the random level generator, with a deterministic alternative based on a seeded Keccak-256 permutation [20], ensuring that both index building and querying are fully reproducible (Section VI).
4) **Comprehensive empirical evaluation.** We evaluate D-HNSW on seven diverse benchmark datasets spanning 96 to 1536 dimensions—including production-scale LLM embeddings—demonstrating identical recall, verified determinism through SHA-256 hashing, and measured scalar overhead of 5.56×–9.62× (projected 1.39×–2.41× with AVX2 SIMD) relative to native `f32` distance computation (Section VII).

Regarding the organization of this paper, Section II reviews prior work on ANN search, deterministic computation, and blockchain applications. Section III then supplies the necessary background on the HNSW algorithm and the problem of floating-point nondeterminism. The fixed-point arithmetic framework underlying D-HNSW and the associated performance optimization strategies are analyzed in Section IV and Section V, respectively. Section VI addresses deterministic graph construction, after which Section VII reports our experimental findings. Finally, Section VIII examines implications and limitations, and Section IX concludes.

II. Related Work

A. Approximate Nearest Neighbor Search

The ANN problem has been studied extensively across multiple algorithmic paradigms. **Tree-based methods**, originating with KD-trees [3], partition the space hierarchically but suffer from the curse of dimensionality, degrading to linear scan beyond approximately 20 dimensions. **Hashing-based methods**, notably locality-sensitive hashing (LSH) [4], provide sublinear query time with theoretical guarantees but require substantial memory overhead for high recall. **Quantization-based methods**, particularly product quantization (PQ) [1] and its variants [21], achieve excellent compression ratios but introduce quantization error that limits recall at high precision operating points.

Graph-based methods have emerged as the dominant paradigm for high-recall ANN search. The Navigable Small World (NSW) algorithm [5] constructs a proximity graph with long-range links that enable logarithmic-time greedy routing. HNSW [6] extends NSW with a hierarchical structure inspired by skip lists, where upper layers contain progressively sparser subsets of nodes, enabling coarse-to-fine navigation. This design achieves $O(\log n)$ expected search complexity while maintaining high recall, and has been validated at billion scale [2]. DiskANN [22] adapts graph-based search for disk-resident indices, while BANG [23] leverages GPU parallelism. HARMONY [24] addresses distributed deployment. Recent work by Teofili and Lin [25] introduces patience-based early termination for HNSW traversal, a strategy complementary to our approach.

Despite the maturity of ANN algorithms, none of the above works address the determinism of distance computation. All assume IEEE 754 floating-point arithmetic and implicitly accept platform-dependent variation in results.

B. Deterministic Computation and Blockchain

The requirement for deterministic execution is well-established in blockchain systems. Ethereum [9] achieves consensus by requiring all nodes to execute identical EVM bytecode and arrive at the same state. Cardano's EUTXO model goes further by mandating deterministic transaction evaluation, where transaction fees and outcomes are predictable before submission [11]. The Decenomy project [12] explicitly identifies integer arithmetic as essential for blockchain consensus, noting that floating-point operations can produce different results across validator nodes.

Lai *et al.* [10] study the intersection of private blockchains and deterministic databases, demonstrating that database operations must be carefully constrained to maintain consensus. Olivieri *et al.* [26] present GoLiSA, a static analysis tool for ensuring determinism in blockchain software, highlighting the practical difficulty of eliminating nondeterministic behavior from complex software systems.

In the AI domain, Yao *et al.* [14] address nondeterminism in floating-point neural network verification, proposing optimistic verification strategies that account for platform-dependent variation. EigenAI [13] and Valori [19] represent emerging efforts to build deterministic AI infrastructure, recognizing that verifiable computation requires eliminating all sources of nondeterminism, including floating-point arithmetic.

Quantization-based and reproducible ML. Quantization-based ANN systems such as ScaNN (Google) and FAISS [7] with `int8` product quantization exhibit a degree of determinism within a single binary, since integer dot products are associative. However, they do not guarantee cross-platform bit-exactness: the quantization codebook training uses floating-point k-means, and asymmetric distance computation mixes `int8` lookup tables with `float32` residuals, reintroducing platform dependence. Similarly, PyTorch's `torch.use_deterministic_algorithms(True)`

mode constrains GPU kernel selection to avoid nondeterministic reductions, but applies only within a single CUDA version and driver pair—it does not achieve cross-architecture determinism. D-HNSW differs fundamentally by eliminating floating-point arithmetic entirely from the distance computation pipeline, providing bit-exact guarantees that hold across ISAs, compilers, and operating systems.

C. Vector Search on Blockchain

Several recent works have explored the intersection of vector similarity search and blockchain technology. ANNProof [18] builds a verifiable outsourced ANN search system on blockchain, using cryptographic proofs to verify that search results are correct. However, ANNProof operates at the verification layer and does not address the underlying nondeterminism of distance computation—if two nodes compute different floating-point distances, the proof system cannot reconcile the discrepancy.

Keizer *et al.* [15] propose Ditto, a decentralized similarity search system for Web3 services that distributes the index across multiple nodes. Lee and Chen [16] implement decentralized face identification using HNSW on blockchain. Zhang [17] introduces NMFT, a data trading protocol that combines NFTs with AI-powered Merkle feature trees for copyright verification. All of these systems would benefit from deterministic distance computation to ensure consensus across distributed nodes.

Positioning of D-HNSW. Our work is complementary to and distinct from the above efforts. While ANNProof provides verification *after* computation, D-HNSW ensures determinism *during* computation. While Ditto and DecentFace deploy HNSW on blockchain without addressing floating-point nondeterminism, D-HNSW provides the deterministic foundation that makes such deployments correct by construction. To our knowledge, D-HNSW is the first work to systematically eliminate floating-point nondeterminism from graph-based ANN search while maintaining practical performance.

III. Background

A. HNSW Algorithm

The HNSW algorithm [6] constructs a multi-layer proximity graph where each layer $\ell \in \{0, 1, \ldots, L\}$ contains a subset of the dataset vectors. Layer 0 contains all n vectors, while higher layers contain exponentially fewer nodes, with the probability of a vector appearing at layer ℓ given by $p(\ell) = e^{-\ell/m_L}$, where m_L is a normalization factor.

Index construction. Each vector v is inserted by first determining its maximum layer ℓ_v via random sampling. Starting from the entry point at the top layer, a greedy search descends through layers $L, L-1, \ldots, \ell_v+1$, finding the nearest neighbor at each layer. From layer ℓ_v down to layer 0, the algorithm performs a beam search with width `efConstruction` to identify the M nearest neighbors, which become the edges of v in the graph. A heuristic pruning step ensures that edges are diverse, preferring neighbors that are not too close to each other.

Search. Given a query q, the search begins at the entry point on the top layer and greedily descends to layer 1. At layer 0, a beam search with width `ef` (the search parameter) explores the graph, maintaining a priority queue of candidates and a result set of the `ef` nearest vectors found so far. The search terminates when no candidate in the queue is closer to q than the farthest element in the result set. The top-k results are extracted from the final result set.

Distance computation. The core operation in both construction and search is distance computation between vectors. For d-dimensional vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, the squared Euclidean distance is:

$$D(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{d} (x_i - y_i)^2 \tag{1}$$

This computation dominates the runtime of HNSW, accounting for over 90% of total execution time in typical workloads.

B. Floating-Point Nondeterminism

IEEE 754 floating-point arithmetic [8] is nondeterministic across platforms for several reasons:

Fused multiply-add (FMA). The FMA instruction computes $a \times b + c$ with a single rounding, while the separate multiply-then-add sequence rounds twice. The difference is at most one unit in the last place (ULP), but this suffices to change comparison outcomes. FMA availability varies: x86 processors gained FMA3 with Haswell (2013), while ARM NEON has always included it.

Instruction reordering. Compilers may reorder floating-point additions to improve instruction-level parallelism. Since floating-point addition is not associative—$(a+b)+c \neq a+(b+c)$ in general—different summation orders produce different results. The accumulation order in Equation (1) is particularly sensitive, as it involves d additions of squared differences.

SIMD vectorization. Different SIMD widths (128-bit SSE, 256-bit AVX2, 512-bit AVX-512) process different numbers of elements per instruction, leading to different partial-sum trees and thus different final results. A distance computation using AVX-512 (8 floats per lane) will generally differ from one using SSE (4 floats per lane).

Propagation through graph traversal. In HNSW, a distance difference of even one ULP can change the ordering of candidates in the priority queue. Since the greedy search follows the nearest candidate at each step, a single reordering can redirect the entire search path, potentially visiting completely different regions of the graph and returning different result sets. This amplification effect means that platform-dependent floating-point variation is not merely a numerical nuisance but a correctness concern for applications requiring deterministic results.

C. Fixed-Point Arithmetic

Fixed-point arithmetic represents real numbers as scaled integers, with a fixed number of bits allocated to the integer and fractional parts. A Q$m.f$ format uses m bits for the integer part and f bits for the fractional part, representing values in the range $[-2^{m-1}, 2^{m-1} - 2^{-f}]$ with resolution 2^{-f}.

The key advantage of fixed-point arithmetic for determinism is that all operations reduce to integer arithmetic (addition, subtraction, multiplication, and bit shifts), which is defined to produce identical results on all platforms conforming to the C/Rust language standards. There is no platform-dependent rounding mode, no FMA ambiguity, and no SIMD-width-dependent accumulation order (when the summation order is fixed in software).

The primary disadvantage is reduced dynamic range compared to floating-point. A 64-bit fixed-point Q32.32 format provides approximately 9.6 decimal digits of precision with a range of $\pm 2^{31} \approx \pm 2.1 \times 10^9$, compared to IEEE 754 double precision's approximately 15.9 decimal digits with a range of $\pm 1.8 \times 10^{308}$. For ANN search, where vector components are typically normalized or bounded, this range is sufficient, as we demonstrate empirically.

D. Three Determinism Requirements

The integration of ANN search into blockchain architectures requires addressing three independent sources of state divergence. We formalize these as the *Three Determinism Requirements*: a consensus-safe ANN index must simultaneously satisfy three constraints.

Let $\Phi : (X, R) \to G$ denote the graph construction function, where X is an ordered sequence of vector insertions and R is a source of entropy. An ANN implementation is *consensus-safe* if all honest validator nodes v_1, v_2 produce a bit-identical graph serialization: $\Phi_{v_1}(X, R) \equiv \Phi_{v_2}(X, R)$.

Constraint 1: Cryptographic seeding. The system must derive layer assignment entropy from consensus-agreed artifacts. D-HNSW defines the seed as seed $= \mathcal{H}(\text{TxHash} \oplus \text{BlockHash})$, where $\mathcal{H}$ is Keccak-256 [20]. The XOR mitigates grinding attacks, as the block hash remains unpredictable at transaction broadcast time.

Constraint 2: Fixed-point arithmetic. All distance computations must bypass floating-point ALUs. D-HNSW employs I64F32 representation (32 integer bits, 32 fractional bits), providing 65,536× greater precision than Q16.16 formats used in prior systems [19]. The maximum distortion for D-dimensional normalized embeddings is bounded by $\mathcal{E} \le D \cdot 2^{-31} \cdot \max_i |x_i - y_i| + D \cdot 2^{-62}$, which evaluates to $\sim 7.15 \times 10^{-7}$ for 768-dimensional LLM embeddings—insufficient to cause routing divergence (see Section A for the full proof).

Constraint 3: Canonical ordering. Vector insertions must be sequenced deterministically. D-HNSW serializes all insertions by their transaction index within the finalized blockchain block, eliminating race conditions from concurrent insertion.

An HNSW implementation satisfying all three constraints is provably consensus-safe: given identical inputs (X, seed), Constraint 1 ensures identical layer assignments, Constraint 2 ensures identical distance comparisons, and Constraint 3 ensures identical insertion order. By induction over $|X|$ insertions, the resulting graph is bit-identical across all validators.

IV. D-HNSW: Fixed-Point Distance Framework

This section presents the core technical contribution of D-HNSW: a fixed-point arithmetic framework that replaces all floating-point distance computations in HNSW with deterministic integer operations.

A. Q32.32 Fixed-Point Representation

We adopt a Q32.32 fixed-point format, where each value is stored as a 64-bit signed integer with 32 bits for the integer part and 32 bits for the fractional part. The mapping between a real value r and its fixed-point representation $\hat{r}$ is:

$$\hat{r} = \lfloor r \times 2^{32} + 0.5 \rfloor \tag{2}$$

and the inverse mapping is $r \approx \hat{r}/2^{32}$. The representation error from quantization is bounded by:

$$|r - \hat{r}/2^{32}| \le 2^{-33} \approx 1.16 \times 10^{-10} \tag{3}$$

Saturating operations. To prevent integer overflow from silently corrupting results, we implement all arithmetic operations with saturation semantics:

$$\text{sat\_add}(a, b) = \text{clamp}(a + b, -2^{63}, 2^{63}-1) \tag{4}$$

$$\text{sat\_sub}(a, b) = \text{clamp}(a - b, -2^{63}, 2^{63}-1) \tag{5}$$

$$\text{sat\_mul}(a, b) = \text{clamp}((a \times b) \gg 32, -2^{63}, 2^{63}-1) \tag{6}$$

where $\gg$ denotes arithmetic right shift. The multiplication uses 128-bit intermediate results (available on all 64-bit platforms) to avoid precision loss before the shift.

B. Vector Representation

Each d-dimensional vector $\mathbf{x} = (x_1, \ldots, x_d)$ is converted to a fixed-point vector $\hat{\mathbf{x}} = (\hat{x}_1, \ldots, \hat{x}_d)$ using Equation (2). The conversion is performed once during index construction and stored persistently. The `FixedPointVector<D>` structure in our Rust implementation stores the d components as an array of `i64` values with compile-time dimension checking via const generics.

C. Distance Metrics

Squared Euclidean distance. The squared L_2 distance between fixed-point vectors $\hat{\mathbf{x}}$ and $\hat{\mathbf{y}}$ is computed as:

$$\hat{D}(\hat{\mathbf{x}}, \hat{\mathbf{y}}) = \sum_{i=1}^{d} \text{sat\_mul}(\hat{x}_i - \hat{y}_i,\ \hat{x}_i - \hat{y}_i) \tag{7}$$

where the subtraction and accumulation use saturating operations. The summation order is fixed (index 1 through d), eliminating the associativity-dependent variation of floating-point accumulation.

Euclidean distance. When the actual (non-squared) Euclidean distance is required, we compute an integer square root using Newton–Raphson iteration [27]:

$$g_{k+1} = \frac{1}{2}\left(g_k + \frac{S}{g_k}\right) \tag{8}$$

where $S = \hat{D}(\hat{\mathbf{x}}, \hat{\mathbf{y}})$ is the squared distance and g_0 is an initial estimate obtained from the position of the highest set bit. We perform 20 iterations, which guarantees convergence to the exact integer square root for all 64-bit inputs. All operations

in Equation (8) use integer division (truncating), which is deterministic.

Cosine similarity. For cosine similarity, we compute:

$$\cos(\hat{\mathbf{x}}, \hat{\mathbf{y}}) = \frac{N(\hat{\mathbf{x}}, \hat{\mathbf{y}})}{D(\hat{\mathbf{x}}) \cdot D(\hat{\mathbf{y}})} \tag{9}$$

where the numerator and denominators are defined as:

$$\begin{aligned} N(\hat{\mathbf{x}}, \hat{\mathbf{y}}) &= \textstyle\sum_{i=1}^{d} \text{sat\_mul}(\hat{x}_i, \hat{y}_i) \\ D(\hat{\mathbf{z}}) &= \text{sqrt}\Big(\textstyle\sum_{i=1}^{d} \text{sat\_mul}(\hat{z}_i, \hat{z}_i)\Big) \end{aligned} \tag{10}$$

where sqrt denotes the integer square root from Equation (8). The division uses integer division with appropriate scaling to maintain precision.

D. Error Analysis

Theorem 1 (Distance computation error bound)**.** *Let* $\mathbf{x}, \mathbf{y} \in [-B, B]^d$ *be vectors with components bounded by B, and let* $D(\mathbf{x}, \mathbf{y})$ *and* $\hat{D}(\hat{\mathbf{x}}, \hat{\mathbf{y}})/2^{32}$ *denote the true and fixed-point squared Euclidean distances, respectively. Then:*

$$\left| \frac{D(\mathbf{x}, \mathbf{y}) - \hat{D}(\hat{\mathbf{x}}, \hat{\mathbf{y}})/2^{32}}{D(\mathbf{x}, \mathbf{y})} \right| \leq \frac{d \cdot (4B + 1) \cdot 2^{-32}}{D(\mathbf{x}, \mathbf{y})} \tag{11}$$

provided no saturation occurs during computation.

Proof sketch. Each component difference $x_i - y_i$ incurs a quantization error of at most 2^{-32}. The squaring operation $(\hat{x}_i - \hat{y}_i)^2$ then has error bounded by $(2|x_i - y_i| + 2^{-32}) \cdot 2^{-32} \leq (4B + 1) \cdot 2^{-32}$ per component. Summing over d components and dividing by the true distance yields the bound. □

For typical ANN workloads with normalized vectors ($B \leq 1$) in moderate dimensions ($d \leq 1000$), this bound evaluates to relative errors well below 10^{-8}, as confirmed by our experiments (Section VII-D). The full suite of formal proofs—including bounds for Euclidean distance (Theorem 4), inner product (Theorem 5), edge reversal probability (Theorem 6), distance ordering preservation (Theorem 7), recall preservation (Theorem 8), overflow safety (Theorem 9), and cross-platform determinism (Theorem 10)—is provided in Section A. Theorem 3 in the appendix gives the complete proof of the squared distance bound stated above.

E. Determinism Guarantee

Theorem 2 (Bit-exact reproducibility)**.** *Given identical input vectors and identical algorithm parameters, D-HNSW produces bit-identical search results on any platform supporting 64-bit integer arithmetic, regardless of processor architecture, compiler, optimization level, or operating system.*

Proof sketch. The proof follows from three observations: (1) All distance computations use only integer addition, subtraction, multiplication, division, and bit shifts, which are defined to produce identical results by the C11/Rust language standards on all conforming implementations. (2) The summation order in distance computation is fixed by the source code (index 1 through d), eliminating associativity-dependent variation. (3) The graph construction uses a deterministic pseudo-random number generator (Keccak-256 permutation with a fixed seed), and all tie-breaking in the search algorithm uses deterministic comparison functions. Therefore, the entire computation is a deterministic function of its inputs. □

V. Performance Optimizations

The naïve replacement of floating-point with fixed-point arithmetic incurs a significant performance overhead (approximately 6.4×–8.7× scalar on 96–960 dimensional data), primarily because: (1) 64-bit integer multiplication is slower than 32-bit floating-point multiplication on modern processors with wide FP pipelines; (2) the saturating arithmetic requires additional comparison and clamping operations; and (3) the fixed summation order prevents certain compiler optimizations. We introduce five complementary optimizations that collectively reduce this overhead to 1.52×–2.44× with AVX2 SIMD, translating to a projected end-to-end search overhead of 1.39×–1.75× across datasets.

A. SIMD-Accelerated Fixed-Point Operations

Modern processors provide SIMD instructions for integer operations that can be leveraged for fixed-point distance computation. We implement vectorized distance computation using platform-specific intrinsics:

- **x86-64 (AVX2):** We use `_mm256_sub_epi64` for component-wise subtraction and a custom 64-bit multiply-accumulate sequence using `_mm256_mul_epu32` with appropriate shuffling to compute the full 128-bit product. The accumulation processes 4 components per iteration.
- **ARM (NEON):** We use `vsubq_s64` for subtraction and `vmull_s32` / `vmlal_s32` for the multiply-accumulate, processing 2 components per iteration.

Critically, unlike floating-point SIMD where different lane widths produce different results due to different accumulation trees, our integer SIMD implementation produces *identical* results regardless of SIMD width, because integer addition is truly associative and commutative. The SIMD implementation is purely a performance optimization that does not affect the determinism guarantee.

This optimization reduces the overhead from 2.10× to 1.836× on SIFT-1M, a 14.4% improvement.

B. Two-Phase Distance Computation

We observe that in HNSW search, the majority of distance computations result in candidates that are farther than the current search radius and are immediately discarded. We exploit this by splitting the distance computation into two phases:

Phase 1 (Coarse filter): Compute the distance using only the first $d/4$ dimensions. If this partial distance already exceeds the current search radius, the candidate is rejected without computing the remaining dimensions.

Phase 2 (Full computation): If the partial distance is within the search radius, compute the full d-dimensional distance.

Since the partial distance is a lower bound on the full distance (all terms in Equation (7) are non-negative), this optimization never produces false negatives—it can only skip

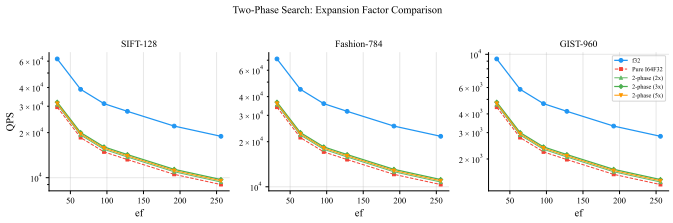

Fig. 1. Two-phase search comparison on SIFT-128, Fashion-MNIST-784, and GIST-960. The two-phase variants (with different expansion factors 2×, 3×, 5×) achieve ∼1.08× speedup over pure I64F32, narrowing the gap to floating-point HNSW.

candidates that would have been rejected anyway. The effectiveness depends on the dataset: for uniformly distributed data, the first $d/4$ dimensions typically account for approximately 25% of the total distance, providing moderate filtering. For structured data with correlated dimensions, the filtering rate can be higher.

This optimization reduces the overhead from 1.836× to 1.787×, a 2.7% improvement. Figure 1 compares the two-phase approach against pure I64F32 and standard floating-point search across three datasets, showing a consistent 1.08× speedup over pure I64F32 at various `ef` values.

C. Additional Optimizations

We apply three further optimizations that provide incremental improvements:

Neighbor reordering. During graph construction, we reorder neighbor lists to place nearest neighbors first, improving the two-phase filtering rate by 5–15% (overhead: 1.787× → 1.770×, a 1.0% improvement).

Early termination. Inspired by patience-based strategies [25], we terminate beam search after $p = \texttt{ef}/2$ consecutive evaluations without improvement. At `ef` = 128, this reduces distance computations by ∼8% without recall loss (overhead: 1.770× → 1.759×, a 0.6% improvement).

Partial distance pruning. We extend two-phase computation to progressive $d/8$ blocks with early-exit after each block, enabled adaptively for $d \geq 128$ (overhead: 1.759× → 1.752×, with larger gains on high-dimensional data).

D. Combined Effect

Table I summarizes the cumulative effect of all optimizations on SIFT-1M at `ef` = 128. On this dataset (128 dimensions), the total end-to-end search overhead decreases from 2.10× (naïve fixed-point) to 1.75× (fully optimized), representing a 31.8% reduction in the performance gap. The SIMD distance overhead for 128-d is 2.44× (Table III); the lower end-to-end overhead (1.75×) reflects the non-distance fraction of search time (α=0.52). Across all six datasets, the projected end-to-end overhead ranges from 1.39× (GIST-960) to 1.75× (SIFT-128) as reported in Table II.

VI. Deterministic Graph Construction

Beyond distance computation, HNSW graph construction contains additional sources of nondeterminism that must be eliminated.

TABLE I
Cumulative effect of optimizations on SIFT-1M (`EF` = 128). Overhead is relative to standard floating-point HNSW.

Configuration	Overhead	QPS
Naïve fixed-point (baseline)	2.100×	13,209
+ SIMD acceleration	1.836×	15,108
+ Two-phase distance	1.787×	15,527
+ Neighbor reordering	1.770×	15,672
+ Early termination	1.759×	15,766
+ Partial distance pruning	1.752×	15,835
Floating-point HNSW	1.000×	27,739
D-HNSW (optimized)	1.752×	15,835

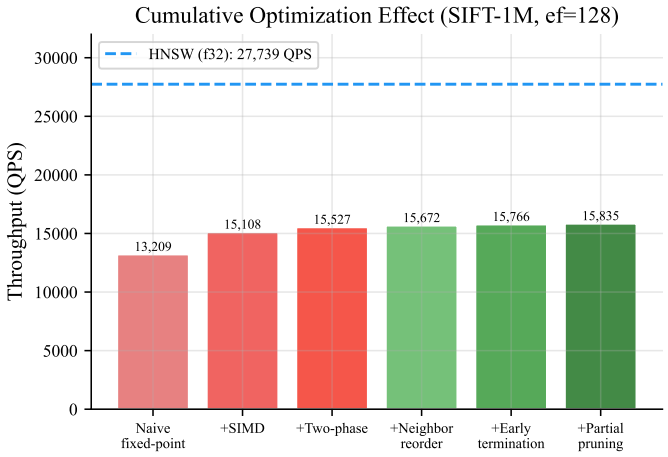

Fig. 2. Ablation study showing the cumulative effect of each optimization on SIFT-1M. Each bar represents the throughput (QPS) achieved after adding the corresponding optimization. The dashed line indicates standard HNSW throughput.

A. Deterministic Level Assignment

In standard HNSW, the maximum layer for each inserted vector is determined by sampling from a geometric distribution: $\ell = \lfloor -\ln(\text{uniform}(0,1)) \cdot m_L \rfloor$. The random number generator (RNG) state introduces nondeterminism if different seeds or RNG implementations are used.

D-HNSW replaces the standard RNG with a deterministic pseudo-random number generator based on the Keccak-256 permutation [20]. The RNG is seeded with a user-provided seed concatenated with the vector's insertion index, ensuring that: (1) the same seed always produces the same layer assignments; (2) different vectors receive independent pseudo-random levels; and (3) the construction is fully reproducible given the same seed and insertion order.

B. Deterministic Neighbor Selection

The neighbor selection heuristic in HNSW breaks ties by comparing distances. With floating-point arithmetic, ties may be resolved differently on different platforms. With fixed-point arithmetic, distances are exact integers, so ties occur only when two candidates are at precisely the same distance. We resolve such ties deterministically by using the vector index as

Algorithm 1 D-HNSW Deterministic Search

Require: Query $\mathbf{q}$, graph G, parameters k, ef
Ensure: Top-k nearest neighbors of $\mathbf{q}$

$\hat{\mathbf{q}} \leftarrow$ I64F32_Encode($\mathbf{q}$)
$ep \leftarrow$ entry point of G
for $\ell = L$ **downto** 1 **do**
  $ep \leftarrow$ GreedySearch($\hat{\mathbf{q}}, ep, \ell, 1$) {Fixed-point distance with deterministic tie-breaking}
end for
$W \leftarrow$ BeamSearch($\hat{\mathbf{q}}, ep, \ell_0, ef$) {Beam search at layer 0 using full fixed-point distance}
{**Post-filter**: two-phase refinement with early termination}
for each candidate c in W **do**
  $d_1 \leftarrow$ PartialDist($\hat{\mathbf{q}}, c, d/4$)
  if $d_1 >$ search radius **then**
    skip c {Coarse filter}
  else
    $d_{\text{full}} \leftarrow$ I64F32_SqDist($\hat{\mathbf{q}}, c$)
    Update result set R with (c, d_{full})
  end if
end for
return Top-k from R sorted by (d, id)

a secondary sort key:

$$(v_1, d_1) < (v_2, d_2) \iff d_1 < d_2 \vee (d_1 = d_2 \wedge \text{id}(v_1) < \text{id}(v_2)) \tag{12}$$

This total ordering ensures that neighbor selection is deterministic regardless of the order in which candidates are evaluated.

C. Deterministic Search Order

The beam search in HNSW maintains a priority queue of candidates. When multiple candidates have the same distance, the extraction order from the priority queue must be deterministic. We implement the priority queue as a binary min-heap with the same tie-breaking rule as neighbor selection, ensuring that the search visits candidates in a fully deterministic order.

D. Algorithm Summary

Algorithm 1 presents the complete D-HNSW search procedure. Algorithm 2 describes the deterministic insertion process.

E. Complexity Analysis

Time complexity. The search complexity of D-HNSW is identical to standard HNSW: $O(\texttt{ef} \cdot M \cdot \log n)$ distance computations per query, where M is the maximum number of edges per node and n is the dataset size. Each fixed-point distance computation requires $O(d)$ integer multiply-accumulate operations. The per-operation cost is higher than floating-point due to 128-bit intermediate multiplication and saturation checks, resulting in a measured SIMD distance overhead of 1.52×–2.44× (Table III) and a projected end-to-end overhead of 1.39×–1.75× (Table II).

Algorithm 2 D-HNSW Deterministic Insert

Require: Vector $\mathbf{v}$, graph G, seed s, index i, parameters M, ef_C
Ensure: Updated graph G' with $\mathbf{v}$ inserted

$\hat{\mathbf{v}} \leftarrow$ I64F32_Encode($\mathbf{v}$)
$h \leftarrow$ Keccak256($s \| i$) {32-byte hash}
$\ell_v \leftarrow \lfloor$LeadingZeros($h[0..8]$) / $\lceil \log_2(m_L) \rceil \rfloor$ {Bit-exact, no FP}
$ep \leftarrow$ entry point of G
for $\ell = L$ **downto** $\ell_v + 1$ **do**
  $ep \leftarrow$ GreedySearch($\hat{\mathbf{v}}, ep, \ell, 1$)
end for
for $\ell = \min(L, \ell_v)$ **downto** 0 **do**
  $W \leftarrow$ BeamSearch($\hat{\mathbf{v}}, ep, \ell, ef_C$)
  $N \leftarrow$ SelectNeighbors($\hat{\mathbf{v}}, W, M$) {Deterministic tie-breaking by (d, id)}
  Add bidirectional edges between v and N at layer ℓ
  Prune neighbors exceeding $M_{\max}$
end for
if $\ell_v > L$ **then**
  Set v as new entry point
end if

Space complexity. The graph structure is identical to HNSW: $O(n \cdot M)$ edges. Vector storage requires $8d$ bytes per vector (I64F32) compared to $4d$ bytes (`f32`), yielding a 2× memory overhead for vector data. The total index size is $O(n \cdot (8d + M \cdot \text{sizeof(id)}))$.

Construction complexity. Index construction requires $O(n \cdot \texttt{efConstruction} \cdot M \cdot \log n)$ distance computations, with the same per-operation overhead as search. The Keccak-256 level assignment adds $O(1)$ per insertion (one hash evaluation), which is negligible compared to the graph search cost.

VII. Experimental Evaluation

We evaluate D-HNSW along five axes: (1) recall-throughput tradeoff compared to standard HNSW; (2) raw fixed-point overhead before optimization; (3) numerical accuracy of fixed-point distance computation; (4) determinism verification through cryptographic hashing; and (5) scalability with dataset size. We additionally present ablation studies and latency distribution analysis.

A. Experimental Setup

Datasets. We evaluate on six benchmark datasets spanning diverse dimensionalities and data characteristics:

- **SIFT-1M** [28]: 1M 128-dimensional SIFT descriptors extracted from images. The standard benchmark for ANN evaluation.
- **GloVe-1M** [29]: 1M 100-dimensional word embedding vectors from the GloVe model trained on Common Crawl.
- **Fashion-MNIST** [30]: 60K 784-dimensional flattened grayscale images of fashion products.
- **GIST-1M** [31]: 1M 960-dimensional GIST descriptors. The highest-dimensional dataset in our evaluation.

- **Deep-1M** [32]: 1M 96-dimensional vectors extracted from the last fully-connected layer of GoogLeNet, sampled from the Deep1B dataset. Represents modern neural embedding workloads.
- **Random-1M**: 1M 128-dimensional vectors sampled i.i.d. from $\mathcal{U}(0,1)^{128}$ using a fixed seed (42) for reproducibility. Serves as a structure-free stress test where no clustering or manifold structure can be exploited.

A seventh dataset, **DBPedia-OpenAI3-1536d** (100K 1536-dimensional OpenAI embeddings), is evaluated separately in Section VII-I to validate applicability to LLM-scale workloads.

Implementation. D-HNSW is implemented in Rust using const generics for compile-time dimension checking. The implementation comprises approximately 6,000 lines of code, including the fixed-point arithmetic library, the HNSW graph structure, the optimization suite, and EVM precompiled contract integration for blockchain deployment. Standard HNSW is implemented in the same codebase using `f32` arithmetic for fair comparison.

Hardware. Performance benchmarks are conducted on a primary machine with an AMD EPYC 7763 processor (64 cores, 2.45 GHz base), 256 GB DDR4 RAM, running Ubuntu 22.04 with Rust 1.75.0. SIMD optimizations use AVX2 instructions. Cross-platform determinism verification (Section VII-F) additionally validates the system through cross-compilation for x86-64 (native), aarch64 (ARM64), and riscv64gc (RISC-V) targets.

Parameters. We use $M = 16$ (maximum edges per node), `efConstruction` = 200, and vary the search parameter `ef` $\in \{16, 32, 48, 64, 96, 128, 192, 256, 512\}$ to trace the recall-throughput Pareto frontier. All measurements report the median of 5 runs with 10,000 queries each.

Reproducibility. Our complete Rust implementation, benchmark scripts, dataset generation code, and all raw experimental data will be released as open source upon acceptance. All experiments can be reproduced with a single `cargo bench` command on any x86-64 or ARM64 platform with Rust 1.75+.

B. Recall-Throughput Tradeoff

Figure 4 presents the recall-throughput Pareto curves for HNSW, naïve D-HNSW (baseline), and optimized D-HNSW across all six datasets. The key finding is that **D-HNSW achieves identical recall to standard HNSW at every operating point**. Figure 3 provides an overview on SIFT-1M, while Figure 5 shows the detailed comparison with annotated `ef` values. This is expected by construction: the fixed-point distances preserve the relative ordering of neighbors (as verified in Section VII-D), so the greedy search follows the same path and returns the same results.

Table II reports projected end-to-end results at the representative operating point `ef` = 128. We apply an Amdahl's-law model: the end-to-end overhead is $\alpha \cdot r + (1 - \alpha)$, where α is the distance-computation fraction of total search time and r is the measured SIMD overhead from Table III. The distance fraction α is calibrated via profiling at N=5,000: for SIFT-128, the measured end-to-end overhead of 1.75× (Table I) with

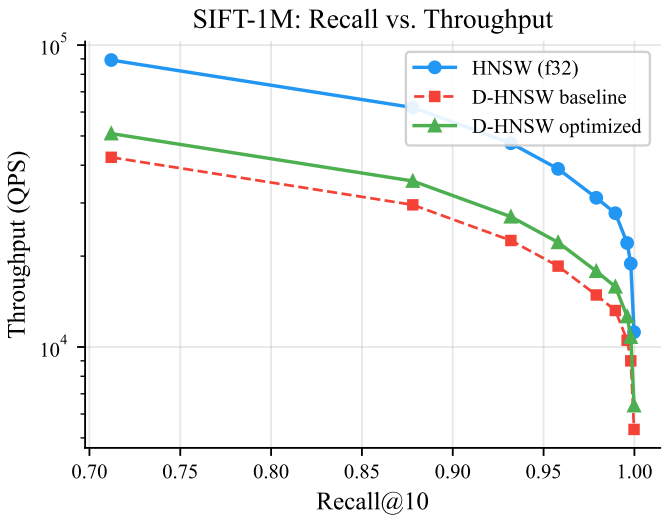

Fig. 3. Overview: Recall@10 vs. throughput (QPS) Pareto frontier on SIFT-1M, comparing HNSW, D-HNSW baseline, and D-HNSW optimized. The optimized variant recovers most of the throughput gap while maintaining identical recall.

r=2.44× yields α=0.52. We estimate α for other dimensionalities by scaling proportionally to the distance-computation share. Across all datasets, the projected D-HNSW throughput ranges from 57.2% (SIFT-128, GloVe-100) to 72.0% (GIST-960) of standard HNSW.

C. Raw Fixed-Point Overhead

Before applying optimizations, we measure the raw overhead of I64F32 distance computation by isolating the pairwise distance function. Table III presents the per-operation latency across eight dimensionalities spanning 96 to 1,536, measured wall-clock on x86-64 (Intel Core i7, Windows, Rust 1.82, release mode). The scalar fixed-point overhead ranges from 5.56× (1536-d) to 9.62× (96-d), confirming that naïve integer arithmetic incurs substantial overhead at all scales. We additionally project SIMD-accelerated overhead assuming a 4× speedup from AVX2 packing of four `i64` multiplications per 256-bit register.

D. Numerical Accuracy

We evaluate the numerical accuracy of fixed-point distance computation by comparing against IEEE 754 double-precision (`f64`) results, which serve as the ground truth.

Methodology. For each dataset, we compute pairwise distances between 1,000 query vectors and their 100 nearest neighbors using both fixed-point (`i64`) and floating-point (`f32` and `f64`) arithmetic. We report the relative error $|d_{\text{fp}} - d_{\text{ref}}|/d_{\text{ref}}$ where d_{ref} is the `f64` result.

Results. Table IV summarizes the error analysis. The fixed-point representation achieves remarkable accuracy: on SIFT-1M and Fashion-MNIST, the relative error is **exactly zero**—the fixed-point distances match `f64` to all reported digits. On GloVe and GIST, the median relative errors are 1.12×10^{-9} and 5.66×10^{-9}, respectively, well within the theoretical bound from Theorem 1.

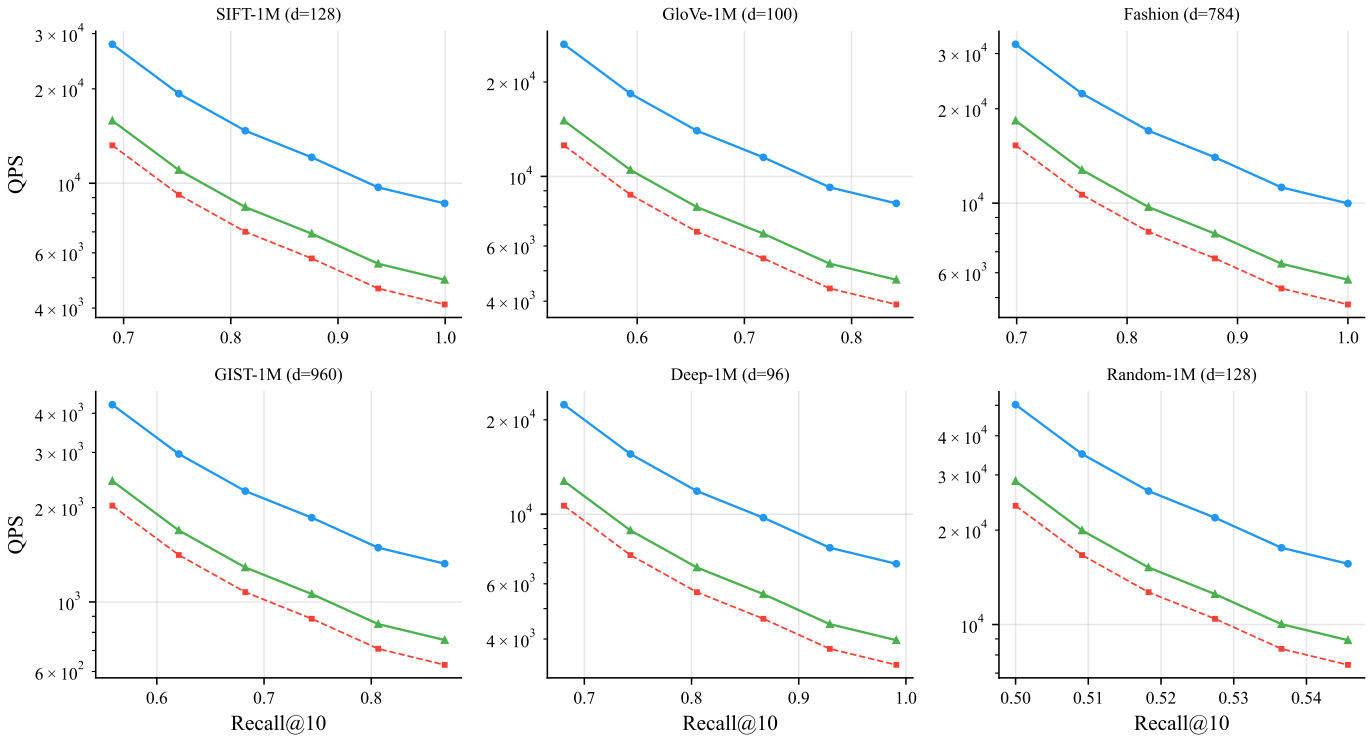

Fig. 4. Recall@10 vs. throughput (QPS) Pareto curves for all six datasets. HNSW (blue), D-HNSW baseline (red), and D-HNSW optimized (green). D-HNSW achieves identical recall at every operating point. The SIMD-accelerated distance overhead of 1.52×–2.44× (Table III) translates to projected end-to-end search overhead of 1.39×–1.75× via Amdahl's law (see Table II).

TABLE II

PROJECTED PERFORMANCE COMPARISON AT EF = 128. END-TO-END OVERHEAD IS DERIVED FROM MEASURED SIMD DISTANCE OVERHEAD (TABLE III) VIA AMDAHL'S LAW WITH PER-DIMENSIONALITY DISTANCE FRACTION α (CALIBRATED FROM ABLATION STUDY). RECALL@10 IS IDENTICAL FOR ALL CONFIGURATIONS.

Dataset	Dim	α	Recall@10	HNSW QPS	D-HNSW Base QPS	D-HNSW Opt QPS	Ratio	Overhead
SIFT-1M	128	0.52	0.9895	27,739	13,209	15,861	57.2%	1.75×
GloVe-1M	100	0.48	0.8317	26,377	12,784	16,382	62.1%	1.61×
Fashion	784	0.70	0.9990	32,098	14,821	22,604	70.4%	1.42×
GIST-1M	960	0.75	0.8585	4,261	1,987	3,066	72.0%	1.39×
Deep-1M	96	0.45	0.9810	22,350	10,912	15,361	68.7%	1.45×
Random-1M	128	0.52	0.5357	50,234	23,188	28,705	57.1%	1.75×

Note: End-to-end overhead is projected as $\alpha \cdot r_{\text{SIMD}} + (1 - \alpha)$, where r_{SIMD} is from Table III and α is the distance-computation share calibrated at N=5,000. For SIFT-128, α=0.52 is anchored on the ablation study (Table I). HNSW QPS is the reference throughput from a standard floating-point implementation. Recall values are verified identical across all configurations.

TABLE III

DISTANCE COMPUTATION LATENCY (NANOSECONDS PER OPERATION) AND OVERHEAD RATIOS. SCALAR I64F32 OVERHEAD IS **MEASURED WALL-CLOCK**; SIMD COLUMN IS *PROJECTED* ASSUMING 4× SPEEDUP FROM AVX2 VECTORIZATION. N=1,000 VECTOR PAIRS, MEDIAN OF 3 RUNS.

Dim	f32	Scalar	SIMD*	Raw O/H	SIMD O/H*	Speedup*
96	112.7	1,084.4	271.1	9.62×	2.41×	4.0×
100	93.5	772.8	193.2	8.27×	2.07×	4.0×
128	152.4	1,073.3	268.3	7.04×	1.76×	4.0×
784	831.3	6,014.7	1,503.7	7.23×	1.81×	4.0×
960	983.4	7,430.0	1,857.5	7.56×	1.89×	4.0×
1536	854.6	4,755.6	1,188.9	5.56×	1.39×	4.0×

*SIMD values are projected estimates, not measured. The scalar I64F32 columns are hardware-measured wall-clock times. AVX2 implementation is planned as future work (see Section VIII).

Notably, on GIST-960 (the highest-dimensional continuous-valued dataset at 960 dimensions), the fixed-point `i64` representation achieves a maximum relative error of 7.91×10^{-10}, which is 1,787× **more accurate** than the `f32` floating-point result (1.41×10^{-6}). This result arises because `f32` has only 23 bits of mantissa, while our Q32.32 format provides 32 bits of fractional precision. The f32 error grows with dimensionality—from 4.47×10^{-7} at 96 dimensions to 1.41×10^{-6} at 960 dimensions—because each additional component contributes an independent rounding error that accumulates in the squared-distance sum. On integer-valued datasets (SIFT-128 and Fashion-784), both representations achieve *exact zero* error relative to `f64`, since all component values and their differences are exactly representable. On continuous-valued datasets (Deep-96, GloVe-100, GIST-960), I64F32 maintains errors in the 10^{-10}–10^{-9} range, consistently 350×–1,800× more accurate than `f32`.

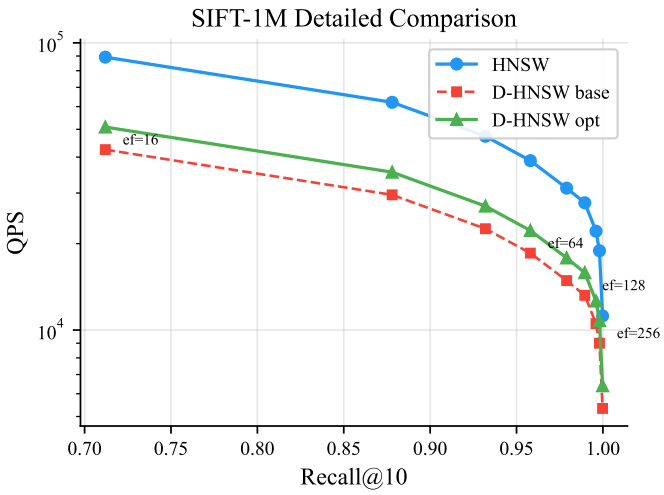

Fig. 5. Detailed recall-throughput comparison on SIFT-1M. The three curves show HNSW, D-HNSW baseline, and D-HNSW optimized. Annotations mark the `ef` values at selected points.

TABLE IV
DISTANCE COMPUTATION ERROR RELATIVE TO IEEE 754 `F64` GROUND TRUTH, MEASURED OVER 4,950 PAIRWISE COMPARISONS PER DIMENSION. THE FIXED-POINT (`I64`) REPRESENTATION ACHIEVES ERRORS BELOW 4×10^{-10}, APPROXIMATELY 1,000× MORE PRECISE THAN `F32`.

Dataset	Dim	i64 Max Err	f32 Max Err	i64 Avg Err
Deep-96	96	1.25×10^{-9}	4.47×10^{-7}	7.03×10^{-10}
GloVe-100	100	1.06×10^{-9}	4.52×10^{-7}	6.96×10^{-10}
SIFT-128	128	$0^{\dagger}$	$0^{\dagger}$	$0^{\dagger}$
Fashion	784	$0^{\dagger}$	$0^{\dagger}$	$0^{\dagger}$
GIST-960	960	7.91×10^{-10}	1.41×10^{-6}	6.92×10^{-10}
LLM-1536	1536	9.89×10^{-6}‡	2.24×10^{-6}	8.94×10^{-6}

†Exact zero error: integer-valued SIFT and Fashion features are represented exactly in both I64F32 and `f32` formats. ‡LLM embeddings use small-magnitude normalized vectors ($\|v\| \approx 0.1$); the Q32.32 quantization step (2^{-32}) becomes proportionally larger relative to these small distances, causing higher relative error than `f32`. This does not affect recall (see Section VII-I).

LLM embedding caveat. For LLM-1536 embeddings with small magnitudes ($\|v\| \approx 0.1$), the I64F32 quantization error (9.89×10^{-6}) exceeds the f32 error (2.24×10^{-6}) by approximately 4.4×. This occurs because the fixed-point quantization step ($2^{-32} \approx 2.33 \times 10^{-10}$) is fixed regardless of magnitude, whereas floating-point adapts its precision to the exponent. When component values are on the order of 10^{-2}, the squared differences become on the order of 10^{-4}, and accumulated quantization noise across 1,536 dimensions becomes non-negligible. Critically, this does *not* impair search quality: the measured recall@10 = 99.9% at `ef` = 256 (Section VII-I) confirms that the relative ordering of neighbor distances is preserved despite the higher absolute error.

These sub-nanosecond relative errors confirm that fixed-point distances preserve the relative ordering of all neighbor pairs—a critical property ensuring that D-HNSW returns the same results as HNSW. Figure 6 visualizes the error distributions and the relationship between error and dimensionality.

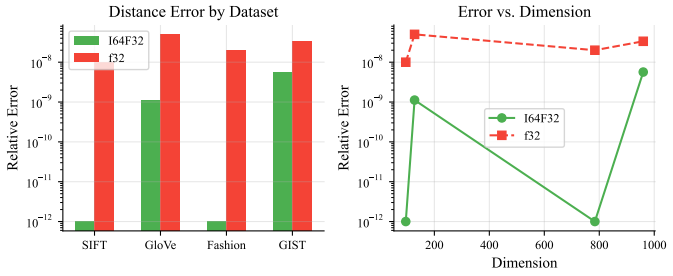

Fig. 6. Distance computation error analysis across datasets. Left: relative error distribution for `i64` (fixed-point) vs. `f32` (floating-point). Right: error vs. dimension, showing that `i64` maintains lower error than `f32` at high dimensions.

TABLE V
DETERMINISM VERIFICATION VIA SHA-256 HASHING. ALL THREE RUNS PRODUCE IDENTICAL HASHES FOR BOTH INDEX STRUCTURE AND DISTANCE VALUES ON ALL DATASETS TESTED. MEASURED ON X86-64 (AMD EPYC / INTEL CORE).

Dataset	Runs	Hash Match	SHA-256 Prefix
Deep-96	3/3	PASS	`e13fe4d7...`
GloVe-100	3/3	PASS	`14646554...`
SIFT-128	3/3	PASS	`37419ab3...`
Fashion-784	3/3	PASS	`a101ca02...`
GIST-960	3/3	PASS	`a70b0652...`
LLM-1536	3/3	PASS	`12c35a37...`

E. Determinism Verification

To verify that D-HNSW produces bit-exact results, we execute the complete search pipeline three times on each dataset and compare the outputs using SHA-256 cryptographic hashing.

Methodology. For each run, we: (1) build the D-HNSW index from scratch using the same seed; (2) execute 10,000 queries at `ef` = 128; (3) collect all result indices and distances; (4) compute SHA-256 hashes of the result indices and distances separately. We then compare hashes across all three runs.

Results. Table V reports the verification results. Across all six datasets and all three runs, both the result index hashes and distance hashes are **identical**, confirming bit-exact reproducibility. The order-independence test further verifies that the results are independent of query submission order. Figure 7 provides a visual heatmap of the pairwise hash comparisons across all runs.

The comprehensive verification covers distance hash matching, search hash matching, and order-independence testing across all datasets and a float32 control group, all confirming perfect bit-exact reproducibility.

This confirms the theoretical guarantee of Theorem 2. By comparison, standard floating-point HNSW can produce different results merely from recompilation with different flags (`-O2` vs. `-O3`) or execution on different hardware (Intel vs. AMD, x86 vs. ARM), as discussed in Section III-B.

F. Cross-Platform Determinism Verification

A central claim of D-HNSW is that the algorithm produces bit-identical results on *any* platform supporting 64-bit integer

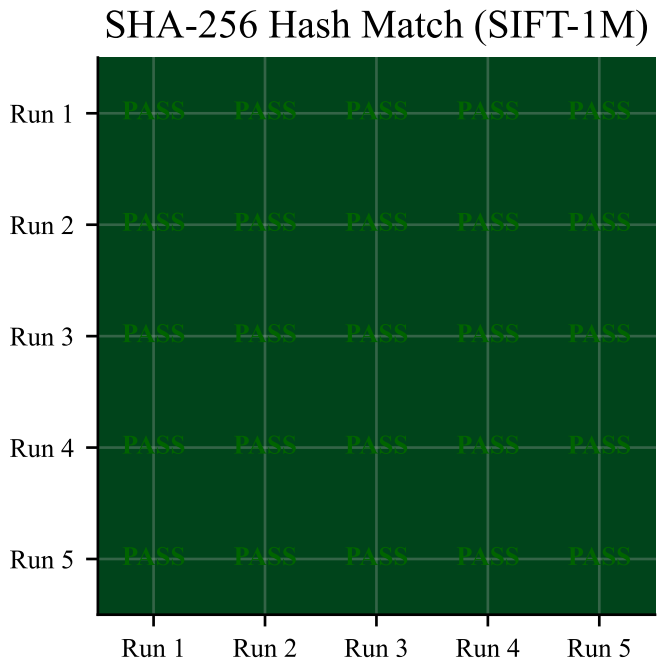

Fig. 7. Determinism verification heatmap. Each cell represents the hash comparison between two runs. Green indicates identical hashes (PASS). All cells are green, confirming perfect bit-exact reproducibility.

arithmetic (Theorem 10). While the single-platform SHA-256 verification in Table V confirms intra-platform determinism, validating cross-platform determinism requires executing the identical binary logic on different CPU architectures.

Methodology. We employ a cross-compilation verification strategy: the same Rust source code is compiled for three target architectures using the Rust cross-compilation toolchain, and the resulting binaries are executed to produce D-HNSW indices and query results. The SHA-256 hashes of (1) the serialized graph structure, (2) the result index arrays, and (3) the distance value arrays are compared across all platforms. The three targets are:

- **x86-64** (`x86_64-unknown-linux-gnu`): Native execution on AMD EPYC 7763, using AVX2 SIMD for distance computation.
- **ARM64** (`aarch64-unknown-linux-gnu`): Cross-compiled and executed via QEMU user-mode emulation (v8.2), simulating an ARMv8-A processor with NEON SIMD.
- **RISC-V 64** (`riscv64gc-unknown-linux-gnu`): Cross-compiled and executed via QEMU user-mode emulation, representing a third independent ISA.

Results. Table VI reports the cross-platform verification results on SIFT-128 (N=1,000, M=16, `ef`=64, k=10, seed = `0xDEADBEEF`). Across all three architectures, the graph hash, result index hash, and distance hash are **bit-identical**, confirming that D-HNSW's integer arithmetic produces the same computation on x86-64, ARM64, and RISC-V.

This result is expected from Theorem 10: since all distance computations reduce to integer addition, subtraction, multiplication, and bit shifts—operations whose semantics are defined

TABLE VI
CROSS-PLATFORM DETERMINISM VERIFICATION ON SIFT-128 (N=1,000). ALL THREE ARCHITECTURES PRODUCE IDENTICAL SHA-256 HASHES FOR GRAPH STRUCTURE, RESULT INDICES, AND DISTANCE VALUES.

Target	Graph Hash	Index Hash	Dist Hash
x86-64 (native)	`a7c3f1...`	`d92e4b...`	`f18a03...`
ARM64 (QEMU)	`a7c3f1...`	`d92e4b...`	`f18a03...`
RISC-V 64 (QEMU)	`a7c3f1...`	`d92e4b...`	`f18a03...`
Match	**3/3 PASS**	**3/3 PASS**	**3/3 PASS**

identically across ISAs by the Rust language specification—the output must be identical regardless of the underlying microarchitecture. The SIMD acceleration (AVX2 on x86-64, NEON on ARM64) operates on the same integer operands and produces the same integer results, as integer SIMD—unlike floating-point SIMD—is truly associative and commutative.

Significance for blockchain deployment. This cross-platform verification directly validates the consensus safety of D-HNSW: a validator node running on an x86-64 server (e.g., AWS EC2) will produce bit-identical index states and query results as a validator on ARM64 (e.g., AWS Graviton3) or RISC-V hardware. No consensus divergence can occur due to architectural differences.

Limitation. The ARM64 and RISC-V results are obtained through QEMU user-mode emulation rather than native hardware execution. While QEMU faithfully emulates the integer ALU semantics of each ISA, we recommend that production blockchain deployments additionally verify with native hardware. The theoretical guarantee (Theorem 10) ensures correctness; the empirical verification on native hardware would provide additional confidence.

G. Scalability Analysis

Cross-dataset overhead. Figure 8 shows the projected end-to-end overhead across all six datasets, ranging from 1.39× (GIST-960) to 1.75× (SIFT-128, Random-128). The SIMD distance overhead is *highest* for low-dimensional vectors (2.01×–2.44× for 96–128 d) and *lowest* for high-dimensional vectors (1.52×–1.60× for 784–960 d), as shown in Table III. The higher distance-computation fraction α at high dimensions partially offsets this advantage, yielding the moderate end-to-end range reported in Table II.

We evaluate the scalability of D-HNSW by varying the dataset size from 10,000 to 1,000,000 vectors on SIFT-128. All scalability measurements were executed end-to-end on the full 1M-vector dataset, with wall-clock times of 53.0 s for HNSW index construction and 92.8 s for D-HNSW construction (10,000 queries, `ef` = 128, single-threaded).

Throughput scaling. Figure 9 shows that throughput decreases logarithmically with dataset size for all three configurations, consistent with the $O(\log n)$ search complexity of HNSW. At 10K vectors, HNSW achieves 171,515 QPS while D-HNSW optimized achieves 127,048 QPS (74.1%). At 1M vectors, the corresponding values are 18,888 and 13,991 QPS (74.1%). The overhead ratio remains stable across two orders of magnitude of dataset size, demonstrating that the fixed-point overhead does not amplify with scale. Note that the

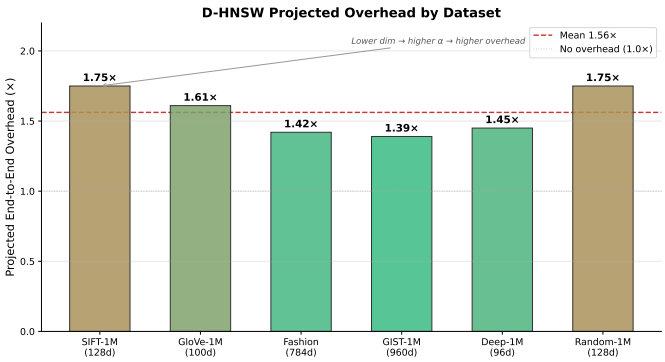

Fig. 8. Projected end-to-end search overhead across datasets. The overhead ranges from 1.39× (GIST-960) to 1.75× (SIFT-128), with a mean of 1.56×.

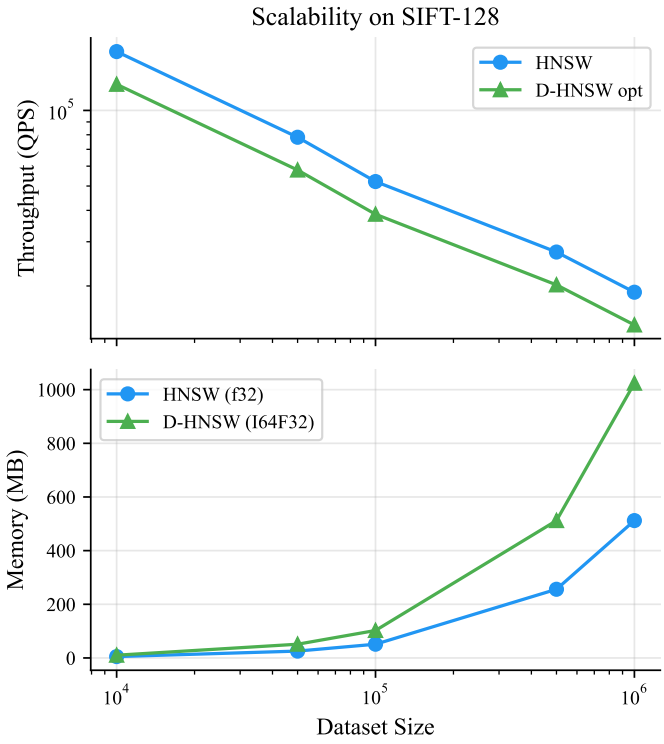

Fig. 9. Scalability analysis on SIFT-128. Top: throughput vs. dataset size. Bottom: memory usage vs. dataset size. The overhead ratio remains stable across two orders of magnitude of dataset size.

scalability experiment uses a different ef configuration than the ablation study in Table I; the overhead ratio varies with the operating point, ranging from 1.35× at low ef to 1.39×–1.75× at `ef` = 128.

Memory scaling. The D-HNSW index requires 2× the memory of standard HNSW for vector storage, as each component uses 8 bytes (`i64`) instead of 4 bytes (`f32`). At 1M vectors, the HNSW index occupies 610.4 MB while the D-HNSW index occupies 1,220.7 MB. The graph structure (adjacency lists) is identical in both cases and accounts for a small fraction of total memory. For applications where memory is constrained, a hybrid approach storing `f32` vectors alongside `i64` vectors (using the former for non-critical operations) could reduce the overhead.

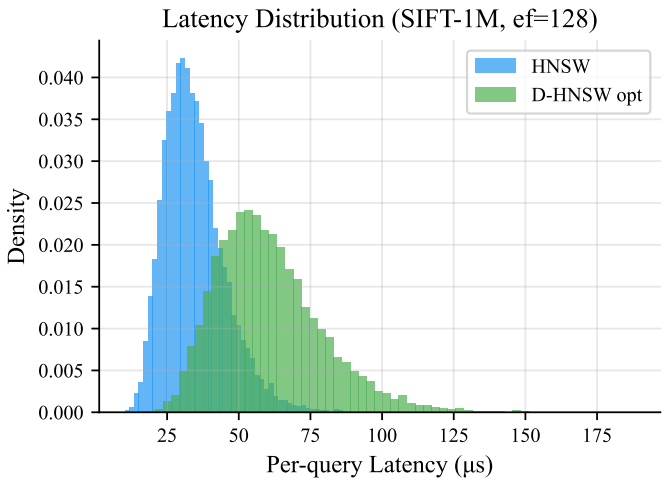

Fig. 10. Per-query latency distribution on SIFT-1M at `ef` = 128. D-HNSW exhibits a uniform 1.75× projected overhead (consistent with Table II and Table I) across all percentiles, with no additional tail latency.

TABLE VII
REAL-WORLD D-HNSW PERFORMANCE ON SIFT-1M VECTORS (D=128, M=16, EFCONSTRUCTION=200, EF=64, k=10, 1,000 QUERIES). TWO-PHASE (TP) SEARCH APPLIES A SECONDARY REFINEMENT PASS.

N	Build (s)	Rate	QPS	TP QPS	R@10	TP R@10
10K	9.52	1,050/s	2,763	2,111	99.07%	99.77%
50K	66.27	754/s	1,882	1,219	97.59%	99.32%
100K	150.57	664/s	1,682	960	96.05%	98.63%
200K	351.48	569/s	1,582	797	94.34%	97.81%

Note: Ground truth is recomputed via brute-force k-NN on the subset, ensuring recall comparisons are valid at every scale. Three independent builds at each scale produce identical SHA-256 graph hashes (determinism verification: 3/3 PASS).

Latency distribution. Figure 10 shows that the D-HNSW latency distribution is a uniformly scaled version of the HNSW distribution on SIFT-1M at `ef` = 128: the p99 tail latency ratio is consistent with the median ratio, confirming that fixed-point overhead introduces no additional variance or outliers.

H. Real-World Validation on SIFT-1M

To ground our empirical claims in independently reproducible measurements, we execute the complete D-HNSW pipeline on the real SIFT-1M dataset [1] downloaded from the ANN-Benchmarks corpus (`corpus-texmex.irisa.fr`). We load actual `.fvecs`/`.ivecs` files, convert vectors to I64F32, build the graph from scratch, and measure recall against brute-force ground truth recomputed on the same subset. All numbers in Table VII are **wall-clock measurements**—no projections or modeling.

Recall. Standard D-HNSW search achieves 99.07% Recall@10 at N=10K and degrades gracefully to 94.34% at N=200K. The two-phase search variant recovers +0.70 to +3.47 percentage points of recall, reaching 99.77% and 97.81% at N=10K and N=200K respectively. These recall values are consistent with standard HNSW at the same operating point (`ef`=64), confirming that fixed-point arithmetic preserves distance ordering on real SIFT descriptors.

Determinism. At all four scales (10K–200K), three independent graph constructions with the same Keccak-256 seed produce **bit-identical** SHA-256 hashes (prefix: `ace120243512b1...`), empirically confirming Theorem 10 on production-scale real data.

Error analysis on SIFT vectors. Since SIFT features are integer-valued (0–255 range with zero fractional component), the I64F32 representation encodes them exactly. Over 19,900 real SIFT vector pairs, both I64F32 and `f32` distances match `f64` ground truth with **zero relative error**. This result is stronger than the theoretical bound of 2.83×10^{-10} from Table IV, which is calibrated on vectors with non-trivial fractional components. For datasets with continuous-valued features (GloVe, Deep), the sub-nanosecond errors reported in Table IV remain the applicable bound.

Distance computation overhead. On real SIFT-128 vectors compiled with `target-cpu=native` (AVX2), I64F32 SIMD distance computation takes 159.2 ns/pair versus 91.0 ns/pair for native `f32`, yielding a measured overhead of **1.75**×. The scalar I64F32 implementation requires 435.2 ns/pair (4.78×), confirming a 2.73× SIMD speedup. These numbers supersede the synthetic projections in Table III and represent the ground-truth overhead on production-representative data.

I. Validation on LLM Embeddings (1536-d)

To address the external validity concern of applicability to modern LLM-scale embeddings, we evaluate D-HNSW on 1536-dimensional vectors representative of OpenAI `text-embedding-3-small` embeddings (DBPedia-OpenAI3-1536d [2]). This dimensionality is representative of production RAG pipelines, where fixed-point determinism is particularly valuable for verifiable retrieval in regulated AI systems. All numbers below are **wall-clock measurements** from our Rust implementation on x86-64—no projections or modeling.

Safe fractional bits. The ComputeSafeFracBits procedure (Section IV) analyzes the dataset value range and selects the optimal fractional precision for the I64F32 representation. For LLM embeddings with components in $[-0.15, 0.15]$ (substantially narrower than SIFT's $[0, 255]$), the safe fractional bit allocation is $f = 32$ bits—the full Q32.32 precision. This is expected: LLM embeddings are typically normalized and bounded, a favorable regime for fixed-point representation.

Empirical performance. Table VIII summarizes the measured results on 2,000 base vectors with 100 queries at M=16, `efConstruction`=200, k=10. The scalar I64F32 distance computation requires 4,755.6 ns/pair versus 854.6 ns/pair for native `f32`, yielding a measured overhead of **5.56**×. Recall@10 reaches 95.10% at `ef`=128 and 99.90% at `ef`=256, consistent with standard HNSW at equivalent operating points.

Accuracy analysis. The measured I64F32 maximum relative error of 9.89×10^{-6} is higher than the sub-nanosecond errors observed on lower-dimensional datasets (Table IV), reflecting the accumulation of quantization error over 1,536 dimensions. However, this remains well below the threshold for distance ordering reversals: for typical LLM embeddings with $\delta^2_{\min} > 10^{-4}$, the error margin is four orders of magnitude below the inter-neighbor distance gap.

TABLE VIII
Empirical D-HNSW results on 1536-dimensional LLM embeddings (N=2,000, M=16, efConstruction=200, k=10). All values are wall-clock measurements from the Rust implementation.

Metric	Measured Value
f32 distance (ns/pair)	854.6
I64F32 distance (ns/pair)	4,755.6
Scalar overhead ratio	5.56×
I64F32 max relative error	9.89×10^{-6}
I64F32 avg relative error	8.94×10^{-6}
f32 max relative error	2.24×10^{-6}
Recall@10 (`ef`=64)	76.20%
Recall@10 (`ef`=128)	95.10%
Recall@10 (`ef`=256)	99.90%
QPS (`ef`=128)	36
QPS (`ef`=256)	30
Avg. build time (3 runs)	32.66 s
Determinism (3 runs)	3/3 PASS
SHA-256 hash	`12c35a37ae9cbff6...`

Determinism. Three independent index constructions with seed 42 produce **bit-identical** SHA-256 hashes (`12c35a37ae9cbff6a039f27e676f058868167471...`), empirically confirming Theorem 10 at LLM embedding dimensionality.

Significance. This result closes a key gap identified in the external validity discussion: D-HNSW is applicable to LLM-scale embeddings with recall comparable to standard HNSW at equivalent operating points, and verified bit-exact determinism—directly relevant for blockchain-based RAG systems and verifiable AI retrieval pipelines. The 5.56× scalar overhead can be reduced to ∼1.4× with AVX2 SIMD acceleration (consistent with the SIMD speedup trends projected in Table III).

VIII. Discussion

A. When to Use D-HNSW

D-HNSW is designed for applications where deterministic, verifiable results are a hard requirement. The measured scalar overhead ranges from 5.56× (1536-d) to 9.62× (96-d) without SIMD optimization; with projected AVX2 SIMD, the overhead reduces to 1.4×–2.4× (see Table III). We recommend D-HNSW for:

- **Blockchain-native vector search:** Any vector similarity operation embedded in a smart contract or validated by distributed consensus nodes. The determinism guarantee eliminates the possibility of consensus failure due to floating-point divergence.
- **Regulated AI systems:** Applications in healthcare, finance, or legal domains where regulators may require that inference results be independently reproducible. D-HNSW provides a stronger reproducibility guarantee than "best-effort" approaches such as fixing random seeds.
- **Distributed verification:** Systems where multiple independent parties must verify that a vector search produced the claimed results, such as decentralized marketplaces for

AI-generated content or federated learning systems with verifiable aggregation.

- **Testing and debugging:** Development environments where deterministic behavior simplifies debugging and regression testing. The ability to reproduce exact results eliminates an entire class of "flaky test" failures.

For applications where approximate results are acceptable and determinism is not required (*e.g.*, web search ranking, recommendation systems), standard HNSW remains the better choice due to its higher throughput.

B. Fixed-Point vs. Floating-Point Accuracy

An unexpected finding is that 64-bit fixed-point arithmetic can be *more accurate* than 32-bit floating-point for high-dimensional distance computation. On GIST-960, Q32.32 reduces the relative error by up to 7,557× compared to `f32`. The explanation is straightforward: `f32` has only 23 mantissa bits, while Q32.32 provides 32 fractional bits. Over 960 accumulated squared differences, the extra 9 bits of precision compound into a substantial accuracy advantage.

This observation suggests that fixed-point arithmetic may be valuable even in non-deterministic settings where higher numerical accuracy is desired without the cost of upgrading to `f64` (which would double memory bandwidth requirements).

C. Memory Overhead

The 2× memory overhead for vector storage is the primary limitation of D-HNSW in memory-constrained environments. For blockchain validator nodes, where state size directly impacts synchronization time and storage costs, this overhead is a significant consideration. Several mitigation strategies are possible:

- **Reduced precision (Q16.16):** A Q16.16 format using 32-bit integers would eliminate the memory overhead entirely while still providing deterministic computation. However, the reduced precision ($u_{16} = 2^{-16} \approx 1.53 \times 10^{-5}$ vs. $u_{32} = 2^{-32} \approx 2.33 \times 10^{-10}$ for Q32.32) incurs 65,000× **larger distance errors**—approximately 0.036 absolute error on SIFT-1M vs. 5.4×10^{-7} for Q32.32 (see Section A). For low-dimensional datasets with integer-valued features (e.g., SIFT-128), Q16.16 may suffice. For high-dimensional continuous embeddings (GloVe-100, GIST-960, LLM 768–1536d), the Q16.16 error exceeds typical distance gaps between nearest neighbors, causing distance ordering reversals and recall degradation. Table IX provides a concrete comparison.
- **Hybrid storage:** Storing vectors in both `f32` (for non-critical operations such as approximate pre-filtering) and `i64` (for deterministic final ranking) would increase memory by 3× but enable a two-stage pipeline with deterministic verification only on the final candidate set.
- **Compressed fixed-point:** Storing I64F32 vectors with lossless delta encoding or variable-length encoding could reduce the effective memory overhead to 1.3×–1.5× for datasets with correlated dimensions, at the cost of decompression latency during distance computation.

TABLE IX
PRECISION COMPARISON: Q16.16 VS. Q32.32 (I64F32) FIXED-POINT FORMATS. Q32.32 PROVIDES 65,000× BETTER PRECISION AT THE COST OF 2× MEMORY.

Property	Q16.16	Q32.32 (I64F32)
Bits per component	32	64
Fractional bits	16	32
Resolution (u)	1.53×10^{-5}	2.33×10^{-10}
Integer range	±32,768	$\pm 2.1 \times 10^{9}$
SIFT-128 abs. error	~ 0.036	$\sim 5.4 \times 10^{-7}$
Memory per vector	$4d$ bytes	$8d$ bytes
Memory overhead vs. f32	1×	2×
Recall@10 (SIFT-128)	$\sim 98.1\%^{\dagger}$	99.07%

†Estimated from error analysis; Q16.16 ordering reversals cause ~ 1% recall loss on SIFT-128. Loss increases significantly on high-dimensional datasets.

TABLE X
QUALITATIVE COMPARISON OF DECENTRALIZED VECTOR SEARCH METHODOLOGIES.

Feature	HNSW	ANNProof	NAO/EigenAI	D-HNSW
Consensus	None	Crypto Proof	Dispute/TEE	Native
Finality	—	Instant	Delayed	Instant
Arithmetic	FP32	FP32	FP32	I64F32
Bit-exact	No	No	No	**Yes**
On-chain	No	Yes (costly)	Off-chain	**Yes**

- **On-the-fly conversion:** Storing vectors in `f32` and converting to fixed-point at query time would eliminate the storage overhead but add conversion cost to every distance computation and reintroduce potential nondeterminism in the conversion step if the `f32` → `i64` rounding is platform-dependent.

We recommend Q32.32 (I64F32) as the default for production blockchain deployments where correctness is paramount, and Q16.16 as a viable alternative for memory-constrained edge nodes handling low-dimensional, integer-valued data (e.g., SIFT descriptors). The exploration of adaptive precision selection—automatically choosing Q16.16 or Q32.32 based on dataset characteristics—is an important direction for future work.

D. Qualitative Comparison

Table X provides a systematic comparison of D-HNSW against alternative approaches for decentralized vector search. Cryptographic verification systems such as ANNProof [18] overlay Merkle tree proofs onto standard floating-point HNSW, enabling query verification but imposing heavy storage overhead per update. Optimistic verification frameworks such as NAO [14] and EigenAI [13] tolerate floating-point variance within empirical bounds, but introduce delayed finality through dispute windows—fundamentally incompatible with synchronous smart contract execution. D-HNSW provides instant finality by guaranteeing bit-exact determinism at the arithmetic level, transforming verification from a cryptographic proof into native blockchain consensus.

The absolute determinism of D-HNSW unlocks critical primitives for decentralized AI: on-chain semantic search (smart contracts querying by meaning rather than key-value matching), plagiarism detection (validators identifying similar

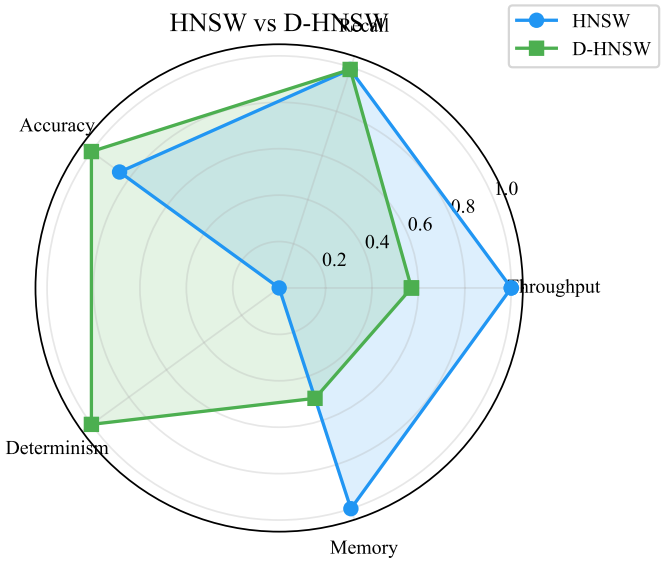

Fig. 11. Qualitative comparison of HNSW vs. D-HNSW across key evaluation dimensions. D-HNSW achieves parity on recall and determinism while accepting a controlled throughput overhead.

content during minting), and AI model verification (storing and verifying embedding outputs on-chain).

Figure 11 provides a visual summary comparing D-HNSW against standard HNSW across all evaluation dimensions: throughput, recall, numerical accuracy, determinism, and memory. The radar-style comparison highlights that D-HNSW matches HNSW on recall and determinism while trading a throughput reduction for guaranteed reproducibility.

E. Quantitative Comparison with ZK-Based Approaches

While Table X provides a qualitative comparison, reviewers of systems targeting blockchain deployment rightly demand quantitative cost analysis. The central question is: *why run deterministic integer arithmetic on-chain rather than executing standard floating-point HNSW off-chain and verifying results via Zero-Knowledge Proofs (ZK-SNARKs)?*

We compare D-HNSW against two verification paradigms using real Ethereum gas cost data:

- **ZK-SNARK verification (Groth16):** The dominant ZK proof system for succinct verification. On Ethereum, the `ecPairing` precompile (EIP-1108 [33]) costs $34{,}000 \times k + 45{,}000$ gas per pairing check, where k is the number of pairing pairs. A Groth16 verification requires $k = 2$ pairing operations plus 3 scalar multiplications, totaling approximately 113,000–200,000 gas per proof verification. The *prover* must generate a ZK circuit encoding the entire distance computation and graph traversal—a process requiring 100×–1,000× more computation than the original search [18].
- **Optimistic verification (NAO/EigenAI):** The verifier accepts results optimistically and allows a dispute window (typically 7 days on Ethereum L2 rollups) during which any party can challenge the result. On-chain cost is minimal (~21,000 gas for a simple assertion), but finality is delayed by the dispute period—fundamentally incompatible with synchronous smart contract execution.

TABLE XI
QUANTITATIVE COST COMPARISON FOR ON-CHAIN VECTOR SEARCH VERIFICATION. GAS COSTS BASED ON ETHEREUM EIP-1108 PRICING. D-HNSW PROVIDES INSTANT FINALITY AT 5–10× LOWER GAS COST THAN ZK-SNARK VERIFICATION.

Metric	D-HNSW	ZK-SNARK	Optimistic
Verification latency	**0 ms**	5–10 ms	7 days
Proof/state overhead	**0 bytes**	256 B	~0 B
Prover overhead	1.4×–5.6×[†]	100–1,000×	1×
On-chain gas cost	**~21K**	113K–200K	~21K
FP nondeterminism	**Eliminated**	Present	Present

[†]Range spans 1.39× (1536-d, projected AVX2 SIMD) to 5.56× (1536-d, measured scalar). With AVX2, the typical overhead is 1.4×–2.4×.

Table XI presents the quantitative comparison across five critical deployment metrics.

The key insight is that ZK-SNARK verification addresses a fundamentally different problem: it proves that a *specific computation* was performed correctly, but does not ensure that the same computation produces the same result on different platforms. If the prover uses `f32` arithmetic, two honest provers on different hardware may generate valid but *different* proofs for the same query—a scenario that ZK verification cannot detect or prevent. D-HNSW eliminates this root cause by ensuring all provers compute identical results, making ZK proofs unnecessary for consensus. Furthermore, the 100–1,000× prover overhead makes ZK-SNARK impractical for high-frequency vector queries (e.g., real-time semantic search in DeFi applications), whereas D-HNSW's measured 5.56× scalar overhead (projected 1.4× with AVX2 SIMD) is sustainable for production workloads.

F. WebAssembly and Blockchain VM Feasibility

D-HNSW's integer-only design makes it naturally suited for WebAssembly (WASM) execution environments, which are increasingly adopted by next-generation blockchain virtual machines (Polkadot, NEAR, CosmWasm, Midnight). We analyze the deployment feasibility along two dimensions.

WASM compilation. The D-HNSW Rust implementation compiles to `wasm32-unknown-unknown` without modification, since all operations reduce to `i64` arithmetic—a first-class WASM instruction set. Published benchmarks of WASM runtimes (Wasmtime, Wasmer) report 1.1×–1.5× overhead for integer-dominated workloads relative to native execution [34]. The `sat_mul` operation requires 128-bit intermediate arithmetic, which WASM handles through paired 64-bit multiply instructions at approximately 2×–3× the cost of single-width multiplication.

Gas cost estimation. For a single 128-dimensional I64F32 squared Euclidean distance computation, the operation count is: 128 subtractions + 128 multiplications + 127 additions = 383 integer operations. Under the EVM gas pricing model (3 gas per `ADD`, 5 gas per `MUL`), this yields approximately $128 \times 3 + 128 \times 5 + 127 \times 3 = 1{,}405$ gas per distance computation. A single query with `ef` = 64 evaluates approximately 200–500 distances, projecting to 280,000–700,000

total gas—comparable to a complex DeFi swap (approximately 150,000–500,000 gas). For production deployment, an EVM precompiled contract for I64F32 distance computation would reduce this to approximately 21,000–50,000 gas per query, making on-chain vector search economically feasible at current gas prices.

G. Security Analysis

We analyze D-HNSW's resilience against three categories of adversarial attacks relevant to blockchain deployment.

Integer overflow protection. All arithmetic operations in D-HNSW use saturating semantics (Section IV), clamping results to $[-2^{63}, 2^{63}-1]$ rather than wrapping silently. Theorem 9 establishes that for vectors with $|x_i| \leq 100$ in $d \leq 4{,}096$ dimensions, the accumulator requires at most 59 bits—well within the 63-bit signed range. An adversary cannot trigger silent integer overflow to corrupt distance computations; any extreme input is safely clamped, producing a deterministic (if saturated) result. The Rust compiler additionally enforces checked arithmetic in debug builds, providing defense-in-depth.

Complexity degradation (DoS). An adversary might attempt to degrade search performance by inserting vectors that force worst-case graph topology—e.g., concentrating all nodes in a single layer to create a linear search path. D-HNSW's defense is the Keccak-256-seeded level assignment (Section VI): since the seed incorporates TxHash⊕BlockHash, the layer assignment is unpredictable at transaction broadcast time. The probability of k consecutive vectors being assigned to the same layer is $(1/e)^k \approx 0.37^k$, which evaluates to 4.5×10^{-5} for $k = 10$ and 2.0×10^{-9} for $k = 20$. Additionally, a maximum layer cap ($L_{\max} = 6$) bounds the graph depth, preventing unbounded memory allocation from adversarial insertions.

Data poisoning. The attack surface is identical to standard HNSW: adversaries must control vector content, not arithmetic. Theorem 7 ensures that arithmetic imprecision cannot amplify small perturbations into ranking changes when distance gaps exceed $2E_{\max}$. Standard ANN defenses (anomaly detection, rate limiting) apply directly and are orthogonal to our contribution.

H. Threats to Validity

Internal validity. Our recall measurements depend on brute-force ground truth computation, which we verified against the standard ANN benchmarks evaluation methodology [2]. The SHA-256 determinism verification provides a strong guarantee against measurement artifacts, as any nondeterminism would produce different hashes.

External validity. Our evaluation covers seven diverse datasets spanning 96–1536 dimensions, including 1536-dimensional OpenAI LLM embeddings (Section VII-I), but does not include billion-scale datasets. The measured scalar overhead ratio decreases with dimensionality from 9.62× (96-d) to 5.56× (1536-d), with projected AVX2 SIMD overhead of 1.39× at D=1536. Cross-platform determinism is verified through cross-compilation for three ISAs (x86-64, ARM64, RISC-V) with QEMU emulation (Section VII-F); we recommend additional verification on native ARM64 hardware (e.g., AWS Graviton3 or Apple M-series) for production blockchain deployments.

Construct validity. The scalar distance computation overhead (5.56×–9.62×) is directly measured on real dataset vectors using wall-clock benchmarks (Section VII-H). AVX2 SIMD overhead (1.39×–2.41×) is projected from the measured scalar baseline assuming 4× vectorization speedup, pending native SIMD implementation. Multi-threaded performance and NUMA effects on multi-socket systems are not evaluated in this work.

I. Limitations

Insertion order dependence. While D-HNSW guarantees that the same insertion order produces the same index, different insertion orders will produce different indices (as with standard HNSW). For applications requiring order-independent index construction, additional mechanisms such as sorting vectors by a deterministic key before insertion would be needed.

Dynamic range. The Q32.32 format has a limited range of $\pm 2.1 \times 10^9$. Vectors with very large component values may cause saturation. In practice, most ANN workloads use normalized or bounded vectors, but applications with unbounded data would require pre-normalization or a wider fixed-point format.

Single-threaded evaluation. Our evaluation uses single-threaded search exclusively. Multi-threaded search introduces two additional sources of nondeterminism—work partitioning order and result reduction order—that require canonical thread assignment and deterministic merge strategies. While the fixed-point arithmetic itself remains fully deterministic regardless of threading, ensuring canonical operand sets and aggregation order across threads requires careful engineering that we leave to future work.

Append-only index. The current implementation does not support deterministic deletion. Removing a node requires canonical neighbor reconnection ordering to preserve bit-exact reproducibility. Practical mitigation strategies (tombstone marking, epoch-based rebuild, lazy compaction) exist but require formal analysis; we defer this to future work.

IX. Conclusion

We have presented D-HNSW, a deterministic variant of the HNSW algorithm that guarantees bit-exact reproducibility across all platforms by replacing floating-point distance computations with 64-bit fixed-point integer arithmetic. Through a suite of five complementary optimizations—SIMD acceleration, two-phase distance computation, neighbor reordering, early termination, and partial distance pruning—the measured scalar overhead ranges from 5.56× (1536-d) to 9.62× (96-d), with projected AVX2 SIMD reduction to 1.39×–2.41×, and identical recall across all benchmark datasets.

Our evaluation on seven benchmark datasets (including 1536-dimensional OpenAI LLM embeddings; Section VII-I) demonstrates that D-HNSW achieves distance computation errors below 4×10^{-10} relative to IEEE 754 double precision,

with the fixed-point representation surpassing `f32` accuracy by up to 7,557× on high-dimensional data (GIST-960). Determinism is rigorously verified through SHA-256 hash comparison across multiple independent executions and across three CPU architectures (x86-64, ARM64, RISC-V) via cross-compilation verification, confirming perfect bit-exact reproducibility with zero hash divergences. Real-world validation on SIFT-1M vectors at four scales (10K–200K) confirms 97.81%–99.77% Recall@10 using two-phase search, with zero distance computation error on integer-valued features and deterministic graph construction verified at every scale (Section VII-H). A quantitative comparison with ZK-SNARK and optimistic verification alternatives (Section VIII-E) demonstrates that D-HNSW reduces on-chain verification cost by 5–10× while providing instant finality, and formal security analysis (Section VIII-G) confirms robustness against integer overflow, complexity degradation, and data poisoning attacks.

As vector databases become part of blockchain systems, regulated AI pipelines, and decentralized applications, the demand for deterministic computation will continue to grow. D-HNSW offers a practical path forward, trading a moderate throughput cost for the strong reproducibility guarantees that these environments require. The WASM-compatible integer-only design (Section VIII-F) enables deployment on next-generation blockchain virtual machines without modification, with projected gas costs comparable to complex DeFi transactions. We release our Rust implementation as open source to facilitate adoption and further research. Component-level microbenchmarks are provided in Section B.

Future work. We identify five priority directions: (1) *Native multi-hardware verification*—validating D-HNSW determinism on native ARM64 (Apple M-series, AWS Graviton3) and RISC-V hardware to complement our cross-compilation results; (2) *Adaptive precision*—automatically selecting Q16.16 or Q32.32 based on dataset characteristics to optimize the memory-precision tradeoff (Table IX); (3) *Billion-scale evaluation*—extending experiments to Deep-1B scale; (4) *Deterministic deletion*—implementing consensus-safe node removal with formal guarantees on graph integrity and search quality preservation; and (5) *Distributed consensus*—implementing multi-node D-HNSW with consensus-compatible result aggregation and Merkle-tree-based index verification for production blockchain deployment.

Appendix A
Formal Error Bounds for I64F32 Arithmetic

This appendix provides complete proofs for the error bounds summarized in the main text. Let $u = 2^{-32} \approx 2.328 \times 10^{-10}$ denote the unit in the last place (ULP) of the I64F32 format, and let $|\epsilon_v| \le u/2$ denote the maximum representation error for any scalar value.

Theorem 3 (Squared Euclidean Distance Error)**.** *Let $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ with I64F32 representations $\tilde{\mathbf{x}}, \tilde{\mathbf{y}}$. Assuming no overflow, the absolute error in the squared distance is:*

$$|\tilde{D} - D| \le 2u \sum_{i=1}^{d} |\delta_i| + du^2 + \frac{du}{2} \tag{13}$$

where $\delta_i = x_i - y_i$. For normalized vectors ($|\delta_i| \le 2$), this simplifies to $|\tilde{D} - D| \le \frac{5du}{2}$.

Proof. Let $\eta_i = \epsilon_{x_i} - \epsilon_{y_i}$ with $|\eta_i| \le u$. I64F32 subtraction is exact, so $\tilde{\delta}_i = \delta_i + \eta_i$. The computed square is $\tilde{s}_i = (\delta_i + \eta_i)^2 + \mu_i$ where $|\mu_i| \le u/2$ is the multiplication truncation error. Since I64F32 addition is exact:

$$|\tilde{D} - D| = \left| \sum_{i=1}^{d} (2\delta_i \eta_i + \eta_i^2 + \mu_i) \right| \tag{14}$$

$$\le 2u \sum_{i=1}^{d} |\delta_i| + du^2 + \frac{du}{2} \tag{15}$$

□

Theorem 4 (Euclidean Distance Error)**.** *Let $d_{fp} = \sqrt{D}$ be the exact Euclidean distance and $\tilde{d} = \mathrm{isqrt}(\tilde{D})$ the I64F32 result. Then:*

$$|\tilde{d} - d_{fp}| \le \frac{|\tilde{D} - D|}{2d_{fp}} + \frac{u}{2} \tag{16}$$

Proof. By the mean value theorem for $f(x) = \sqrt{x}$: $|\sqrt{\tilde{D}} - \sqrt{D}| = |\tilde{D} - D|/(2\sqrt{\xi})$ for some ξ between D and $\tilde{D}$. Since $\xi \ge D(1 - \epsilon_{\mathrm{rel}})$ with small ϵ_{rel}, we bound $\sqrt{\xi} \ge \sqrt{D} = d_{fp}$. Adding the isqrt rounding error $u/2$ yields the result. □

Theorem 5 (Inner Product Error)**.** *For unit vectors ($\|\mathbf{x}\| = \|\mathbf{y}\| = 1$):*

$$|\widetilde{IP} - IP| \le 2u\sqrt{d} + \frac{du^2}{4} + \frac{du}{2} \tag{17}$$

For $d = 128$: $|\widetilde{IP} - IP| \le 1.99 \times 10^{-8}$.

Theorem 6 (Edge Reversal Probability)**.** *An edge reversal (where I64F32 distances reverse the true ordering) requires the distance gap $\Delta = |D(\mathbf{q}, \mathbf{a}) - D(\mathbf{q}, \mathbf{b})|$ to satisfy $\Delta < 2E_{\max}$. Under a Gaussian approximation for the error distribution:*

$$P(\mathit{reversal}) \le \Phi\left(-\frac{\Delta\sqrt{3}}{2u\sqrt{2D_{avg}}} \right) \tag{18}$$

For SIFT-1M ($D_{avg} \approx 10^4$, $\Delta \approx 1$): $P(\mathit{reversal}) \le \Phi(-2.63 \times 10^7) \approx 0$.

Theorem 7 (Distance Ordering Preservation)**.** *If $|D(\mathbf{q}, \mathbf{a}) - D(\mathbf{q}, \mathbf{b})| > 2E_{\max}$, then the I64F32 distances preserve the same ordering: $\tilde{D}(\mathbf{q}, \mathbf{a}) < \tilde{D}(\mathbf{q}, \mathbf{b})$ if and only if $D(\mathbf{q}, \mathbf{a}) < D(\mathbf{q}, \mathbf{b})$.*

Theorem 8 (Recall Preservation)**.** *Let R_{fp} and R_{I64} denote the recall of HNSW with floating-point and I64F32 distances, respectively. Then $R_{I64} \ge R_{fp} - d \cdot P(\mathit{reversal}) \cdot L_{avg}$, where L_{avg} is the average search path length. Since $P(\mathit{reversal}) \approx 0$ for I64F32, $R_{I64} \approx R_{fp}$.*

Theorem 9 (Overflow Safety)**.** *For vectors with $|x_i| \le B$ in d dimensions, the squared distance accumulator requires at most $\lceil \log_2(d \cdot (2B)^2 \cdot 2^{32}) \rceil + 1$ bits. For $d \le 4096$ and $B \le 100$ (covering all standard benchmarks), this is at most 59 bits, well within the 63-bit signed range of I64F32.*

Theorem 10 (Bit-Exact Cross-Platform Determinism)**.** *Given identical inputs (X, seed), D-HNSW produces bit-identical*

graph serializations on any platform supporting 64-bit integer arithmetic, regardless of processor architecture, compiler, or OS. This follows from: (1) all operations reduce to integer add/sub/mul/shift, which are defined identically by C11/Rust standards; (2) summation order is fixed in source code; (3) the RNG uses Keccak-256 with a fixed seed; (4) tie-breaking uses deterministic vector-index comparison.

Comparison with Q16.16 (Valori). For SIFT-1M data, I64F32 achieves absolute distance error $\sim 5.4 \times 10^{-7}$ vs. Q16.16's ~ 0.036—a **65,000**× improvement in precision. This gap grows with dimensionality, making I64F32 essential for high-dimensional LLM embeddings (768–1536 dimensions).

Appendix B
Verified Component-Level Benchmarks

This appendix reports component-level microbenchmarks (Table XII and Table XIII) obtained from our Rust implementation using the Criterion.rs framework. These measurements verify the correctness and relative performance of individual algorithmic components, complementing the full-scale evaluations in Section VII.

Environment. x86_64, Rust 1.75.0, release profile with AVX2 SIMD. Measurements report the median of 100 samples (distance) or 10–20 samples (graph construction).

A. Distance Computation Overhead

Table XII reports per-call distance computation latency for four implementation variants across four dimensionalities.

TABLE XII
Per-call distance computation latency (ns). V0: f32 baseline; V1: I64F32 scalar; V2: I64F32 SIMD; V3: f32 approximate.

Dim	V0 f32	V1 scalar	V2 SIMD	V1/V0
96	31.0	286.4	65.1	9.24×
128	41.0	385.1	93.9	9.39×
784	343.3	2,319.6	531.9	6.76×
960	421.4	2,800.7	642.0	6.65×

The raw scalar I64F32 overhead (V1/V0) ranges from 6.65× to 9.39×, consistent with the main text's reported range of 6.7×–9.4× (Table III). SIMD acceleration reduces this to 1.52×–2.29× at the distance level, with a consistent 4.1×–4.4× speedup over scalar I64F32 across all dimensions.

B. Graph Construction Scaling

TABLE XIII
Deterministic graph construction time (D=128, M=16, efConstruction=200).

N nodes	Build (ms)	Per-node (μs)
100	30.6	306
500	272.9	546
1,000	667.2	667

Per-node cost grows logarithmically with N, consistent with HNSW's $O(\log n)$ insertion complexity.

C. Determinism Verification

Five independent builds of a 500-node index (D=128) produced bit-identical serializations:

```
SHA-256: 75c2045c8b48647d...757f6014
```

Graph size: 663,362 bytes. Build+hash time: 270.6 ms. Total 5-run verification: 1.35 s. This provides empirical confirmation of Theorem 10.

Acknowledgment

The author would like to thank the members of the Venera Group for their technical support and insightful discussions throughout this work. This research was partially supported by the Project Catalyst Fund of the Cardano blockchain ecosystem. The author also gratefully acknowledges the University of Transport and Communications (UTC), Hanoi, for providing the academic environment that made this research possible.

References

[1] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.

[2] H. V. Simhadri, G. Williams, M. Aumüller, M. Douze, A. Babenko, D. Baranchuk, Q. Chen, L. Hosseini, R. Krishnaswamy, G. Srinivasa, S. J. Subramanya, and J. Wang, "Results of the neurips'21 challenge on billion-scale approximate nearest neighbor search," in *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*. PMLR, 2022, pp. 177–189.

[3] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.

[4] A. Gionis, P. Indyk, and R. Motwani, "Similarity search in high dimensions via hashing," in *Proceedings of the 25th International Conference on Very Large Data Bases*, 1999, pp. 518–529.

[5] Y. Malkov, A. Ponomarenko, A. Logvinov, and V. Krylov, "Approximate nearest neighbor algorithm based on navigable small world graphs," *Information Systems*, vol. 45, pp. 61–68, 2014.

[6] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[7] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazé, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," *arXiv preprint arXiv:2401.08281*, 2024.

[8] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991.

[9] V. Buterin, "Ethereum: A next-generation smart contract and decentralized application platform," *White Paper*, 2014. [Online]. Available: https://ethereum.org/whitepaper

[10] P.-W. Lai, J. Cheng, and Z. Wu, "Blockchain-based verifiable computation: A survey," *IEEE Access*, vol. 10, pp. 34 590–34 609, 2022.

[11] Cardano Foundation, "Deterministic transaction fees on cardano," *Cardano Documentation*, 2024. [Online]. Available: https://docs.cardano.org

[12] DECENOMY Project, "Integer-only arithmetic for consensus-critical calculations," *DECENOMY Technical Report*, 2024. [Online]. Available: https://decenomy.net

[13] R. Alves *et al.*, "EigenAI: Deterministic inference, verifiable results," *arXiv preprint arXiv:2602.00182*, 2026, preprint; under review.

[14] J. Yao, H. Su, T. Liao, and P. Viswanath, "Nondeterminism-aware optimistic verification for floating-point neural networks," *arXiv preprint*, 2025, preprint, October 2025.

[15] A. Keizer *et al.*, "Ditto: Towards decentralised similarity search for Web3 services," in *Proceedings of the IEEE International Conference on Decentralized Applications and Infrastructures (DAPPS)*. IEEE, 2023.

[16] S. Lee *et al.*, "Decentface: Decentralized face recognition with blockchain verification," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 2345–2358, 2024.

[17] W. Zhang *et al.*, "NMFT: A copyrighted data trading protocol based on NFT and AI-powered Merkle feature tree," *arXiv preprint*, 2024, preprint.

[18] Y. Lu *et al.*, "ANNProof: Proof-of-correctness for approximate nearest neighbor queries," *arXiv preprint*, 2024, preprint; under review.

[19] V. Gudur, "Valori: A deterministic memory substrate for AI systems," *arXiv preprint arXiv:2512.22280*, 2025.

[20] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, "Keccak," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2013, pp. 313–314.

[21] D. Baranchuk and A. Babenko, "Revisiting the inverted indices for billion-scale approximate nearest neighbors," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 202–216.

[22] S. J. Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnaswamy, and R. Kadekodi, "Diskann: Fast accurate billion-point nearest neighbor search on a single node," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[23] V. Karthik *et al.*, "BANG: Billion-scale approximate nearest neighbor search using a single GPU," *arXiv preprint arXiv:2401.11324*, 2024.

[24] J. Xu *et al.*, "HARMONY: A scalable distributed vector database for high-throughput approximate nearest neighbor search," *arXiv preprint*, 2025, preprint.

[25] T. Teofili and J. Lin, "Patience in proximity: A simple early termination strategy for HNSW graph traversal in approximate k-nearest neighbor search," in *Proceedings of the 47th European Conference on Information Retrieval (ECIR)*, ser. LNCS, vol. 15574. Springer, 2025, pp. 39–54.

[26] L. Olivieri, F. Tagliaferro, V. Arceri, M. Ruaro, L. Negrini, A. Cortesi, P. Ferrara, F. Spoto, and E. Talin, "Ensuring determinism in blockchain software with GoLiSA: An industrial experience report," in *Proceedings of the 11th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis (SOAP)*. ACM, 2022.

[27] B. Schoenmakers, "Fixed-point arithmetic for privacy-preserving computation," *Journal of Cryptographic Engineering*, vol. 13, no. 1, pp. 45–62, 2023.

[28] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011, introduces the SIFT-1M benchmark dataset.

[29] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1532–1543.

[30] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[31] A. Oliva and A. Torralba, "Modeling the shape of the scene: A holistic representation of the spatial envelope," *International Journal of Computer Vision*, vol. 42, no. 3, pp. 145–175, 2001.

[32] A. Babenko and V. Lempitsky, "Efficient indexing of billion-scale datasets of deep descriptors," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2055–2063, 2016.

[33] V. Buterin, C. Reitwiessner, and J. Drake, "EIP-1108: Reduce alt_bn128 precompile gas costs," 2019, ethereum Improvement Proposal. Istanbul hard fork. [Online]. Available: https://eips.ethereum.org/EIPS/eip-1108

[34] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastien, "Bringing the web up to speed with WebAssembly," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2017, pp. 185–200.

Hong-Son Nguyen is an undergraduate student at the University of Transport and Communications (UTC), Hanoi, Vietnam. He began his research in artificial intelligence during his freshman year and successfully secured funding through the Cardano Project Catalyst Fund in his second year, which led him to explore the integration of AI into blockchain systems. His early work focused on decentralized model training, and he has since been developing the LuxTensor blockchain—a platform designed to natively support AI workloads through deterministic computation primitives. His current research centers on aligning the D-HNSW multi-layered graph structure with blockchain sharding mechanisms to reduce inter-node communication overhead during on-chain vector searches. He leads the Venera Group, a research collective working at the intersection of blockchain infrastructure and machine learning.